

7. Achieving Resilience and Event Processing Reliability in Event-Driven Microservices

Hugo Filipe Oliveira Rocha¹

(1) Ermesinde, Portugal

This chapter covers:

- Understanding common failures in distributed microservice architectures
- How event-driven architectures can provide more reliable processes
- Understanding the impacts of the several message delivery semantics and how to apply each of them to event-driven services
- Maintaining correctness when saving state and publishing events
- What we can use from ACID 2.0 and apply to event processing
- How to avoid message leak
- Applying common resilience patterns in event-driven services
- How to repair state in event-driven services
- Using the bulkhead pattern to contain failures

The Banqiao Dam is a major dam in China. After its construction, it suffered from some construction and engineering errors, which were repaired and made the dam undergo a significant redesign. After the new design, it was dubbed the “iron dam” and considered unbreakable. In 1975, typhoon Nina affected the nearby location, and although the hurricane-force winds are arguably the most infamous characteristic of typhoons, another unsung consequence of tropical cyclones is torrential rains. Typhoon Nina stopped moving due to a cold front, which made it pour down torrential rain on the same location for three days. That location was the region of the Banqiao Dam. The immense quantity of water eventually compromised the dam’s integrity and led to a catastrophic fail-

ure. Seven hundred million cubic meters of water flowed through the surrounding areas. The destruction of the dam triggered a domino effect which provoked the collapse of other 61 dams throughout the region. The dam failure triggered a chain reaction that caused the third deadliest flood in world history.

An unavoidable challenge of typical microservice architectures is dealing with cascading failures, which resembles this particular episode of the Banqiao Dam. Sometimes one nonessential and somewhat isolated service can bring the whole architecture down due to the chain of synchronous requests between components. The failure of that inconspicuous service is propagated across the complex network of synchronous calls, eventually spreading the failure to critical components that stop vital business processes.

Event-driven architectures are naturally resilient to these issues due to their asynchronous nature. The event queues between the services are a natural barrier to load peaks and limit direct dependencies between services. One failure in a single service isn't propagated across the architecture; instead, it often affects only that service, endowing the architecture with higher resilience.

However, as with most things in software engineering, it's about trade-offs and deciding what's best on a given use case. As we discussed in Chapter 1, event-driven architectures have their own set of limitations and challenges. One of them is to perceive events as the source of truth. Events aren't simple signals anymore; they carry meaning and are as paramount to the system as databases in traditional applications. Consistency in an event stream is pivotal and as necessary as consistency in a database. Failing a database write in a monolithic application when a customer saves information is as critical as not publishing an event in an event-driven architecture.

Guaranteeing a coherent approach is pivotal to achieve consistency across the architecture. As discussed in Chapter 5, microservice architectures' distributed nature forces us to look at consistency differently. This different view on consistency often translates into relying on a different set of properties we can apply to event-driven architectures. The state can

become invalid due to issues with new releases. In these situations, event-driven services provide a unique and powerful way to recover data by rewinding the event stream to repair state. Resilience also relates to the system's availability and how it deals with its dependencies; patterns like the bulkhead pattern can help provide stability when dependencies fail, which we will discuss in this chapter.

7.1 Common Failures in Microservice Architectures and How They Relate to Event-Driven Architectures

While we are building functionality, we often focus on the solution's design, its reusability, and cleanness. Thinking about how it can go wrong, it's often unnatural, like throwing mud into a pristine masterpiece. We are often inclined to assume things for granted, the database, the event broker, the network, etc. Things are bound to go wrong; in production, nothing is taken for granted. Our applications' success is directly related to its ability to mitigate the impact of the failure. This section will discuss common failures in distributed microservice architectures and how relevant they are in an event-driven architecture.

Let's analyze a typical topology of the services we discussed until now. A regular service has its own physical resources, might have a database, and communicates with an event broker. It communicates to several types of external dependencies through different protocols. Figure [7-1](#) illustrates this topology.

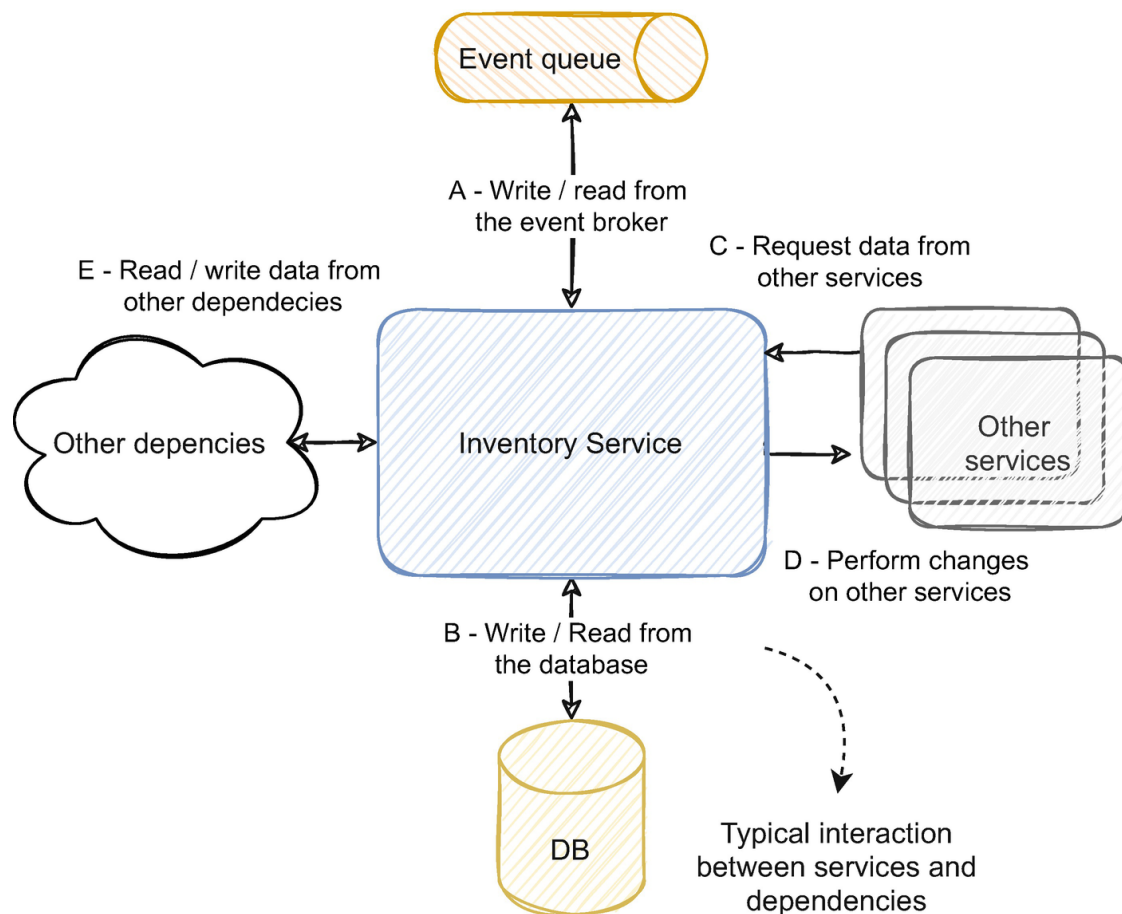


Figure 7-1 Typical service topology and common dependencies

Event-driven services typically interact with the message broker, either by reading or writing events or both (A). They might need to store some data by writing and reading data from a database (B). Although more sparingly, services might synchronously request data from other services through, for example, an HTTP request (C). They also might change data on other services through synchronous requests (D). Besides other services, they might have different dependencies, for example, a distributed cache like Redis (E).

Each of these interactions can fail due to several reasons. The broker might face a catastrophic failure and be unreachable. The same can happen to each of the other services or the database. The service itself can suffer a catastrophic failure due to issues in the underlying hardware. Even if both the service and its dependencies are healthy, the communication between them can fail. We typically write software with unmentioned assumptions like our trust in the network's reliability (illustrated in the fallacies of distributed computing).¹ The network can fail, and it will fail eventually like every other dependency we mention in Figure [7-1](#).

As we said in Chapter [6](#), with enough scale and time, unlikely and far-fetched problems become certain.

Infrastructure components and hardware can fail due to their physical components, although they only affect the applications on rare and isolated occasions. However, network issues like virtual hosts' failure or connectivity issues, like network partitions and similar, are more frequent. Besides physical components failing, maintenance operations can introduce instability; I'm sure we all relate to some kind of patch or upgrade gone wrong. Sparingly the very update of the operating system running the services and upgrading versions can introduce issues, and, of course, human errors while operating these infrastructure components. Other teams operate other services in the architecture; new releases can introduce bugs or make the service unavailable.

Direct dependencies, like synchronous requests, overall increase the likelihood of the system's instability. In the use case of Figure [7-1](#), we depict an inventory service; other services might request data from it, like the order service to fetch the current product's stock. The inventory service might depend on other services, for example, a location service to fetch the warehouses' addresses. The location service itself might depend on other services. Each of those services has its own dependencies, which can all fail and will fail eventually. The overall system's availability is often translated into the combined availability of all services, which is lower and harder to manage. This kind of cascading failure, which we will detail next, is a common failure mode in microservice architectures that event-driven services can mitigate.

7.1.1 Cascading Failures and Event-Driven Services

The services and their dependencies can fail due to many reasons. This failure can make the service unavailable and fail requests. A local failure has limited impact, depending on the service's business function. However, as we discussed with the Banqiao Dam, when that failure triggers a cascading failure to other services in the architecture, the impact grows exponentially and can bring the whole system to a halt. This subsection will work through a practical example of typical microservice architecture and how it would play out in an event-driven architecture.

In typical microservice architectures, communication between services is synchronous, through, for example, HTTP requests. This kind of architecture is prone to develop a complex network of synchronous requests. You might recall the example we mentioned when detailing distributed tracing in Chapter 4, how Uber traces 1000 microservices² and its complex network. In the network, there are large nodes that several services depend upon; if one of these becomes unavailable, it will likely affect the whole architecture. But it can also be the other way around; less critical and more isolated services might cascade issues to other more essential services and aggravate the problem. Let's detail how a cascading failure could affect an eCommerce workflow similar to the ones we discussed until now in a microservice architecture.

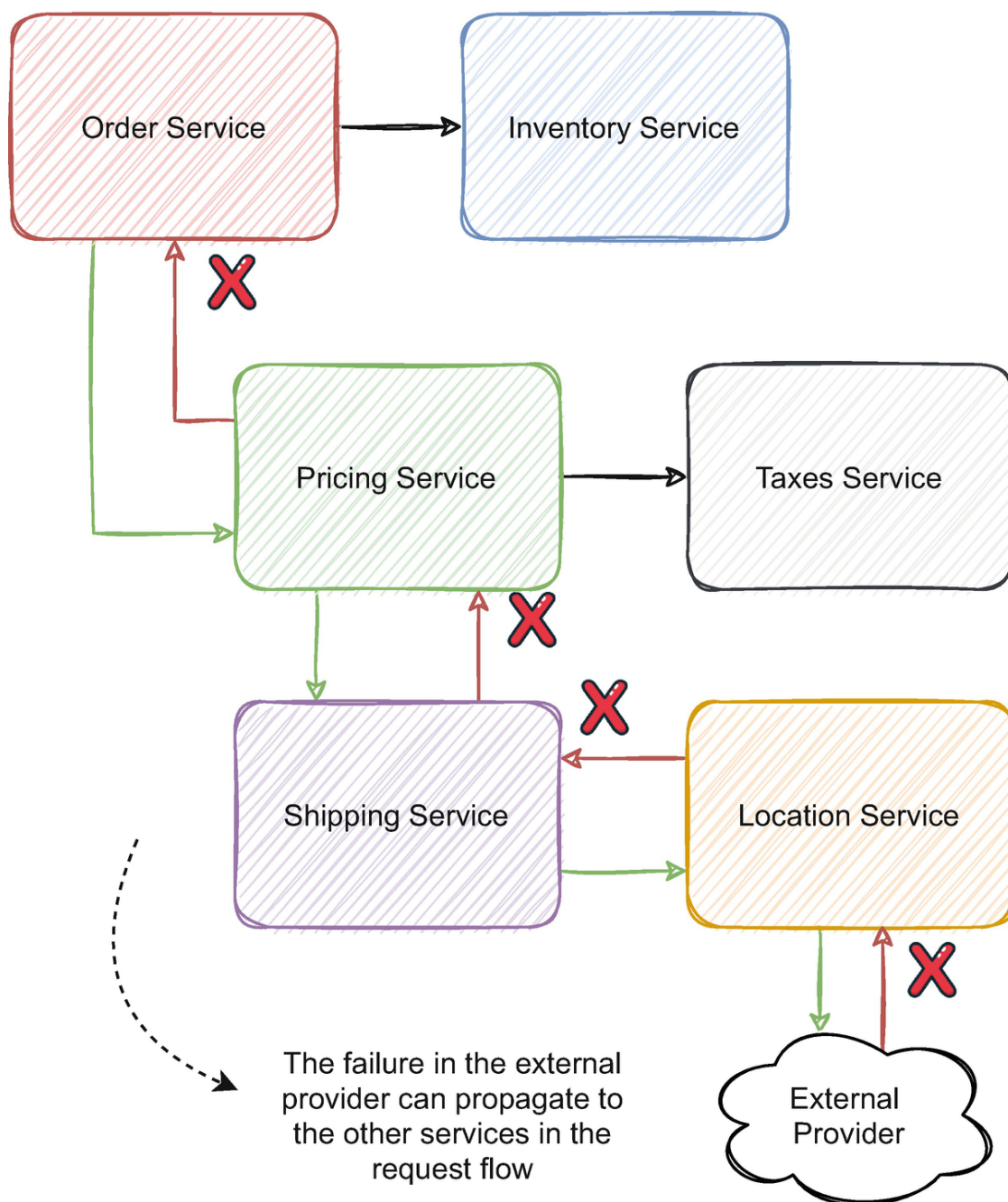


Figure 7-2 A failure in an external provider cascade to the other upstream services

The order fulfillment flow requests the pricing service to calculate the fees for each order it receives. The pricing service depends on the shipping service to process the shipping and address information, which depends on the location service, which manages more generic information about geographic and country locations. The location service also depends on an external provider to fetch the country's information. In the example, the location service fails to fetch data from the external provider. By failing to fetch the information, it will also fail to respond successfully to the shipping service. The failure is cascaded to the other upstream services, eventually failing the order fulfillment process. In this case, a relatively isolated issue, and not very relevant business-wise, ends up affecting the main order fulfillment flow.

In this case, it was an external provider that failed. But it could happen to any of the other service's dependencies we discussed earlier. For example, the database could also fail, and although the database becoming unavailable is relatively uncommon, a load peak can make the database struggle due to limited physical resources. In these cases, a peculiar phenomenon often occurs. A common approach to deal with failing requests is to retry the request. As the service struggling with the excessive load starts to fail some requests, the requesting services also increase their throughput due to the retry strategy. A particular situation where I've experienced this was when a service became unavailable due to connectivity issues. Once the team solved those issues and the service started to recover, the additional retry throughput of every dependency fully occupied the service's resources and crashed the service again due to an overwhelming load. This issue highlighted the need for appropriate coping strategies like backoff retries and circuit breakers (further discussed in the next sections).

Unexpected load affecting services with modest resources is also a common reason for cascading failures. Imagine if, with the same example of Figure [7-2](#), the platform had a higher usage due to a sales season. The order service would likely have a peak in usage, which often translates into higher numbers of requests to the downstream services. If the order service received a peak in orders, it would likely request more information from the pricing service to fulfill those orders. The pricing service

would likely translate that load peak to the shipping service and the shipping service to the location service. The order, pricing, and shipping service would likely have ample resources due to their criticality. Their importance is also obvious in the order fulfillment flow. However, if the location service is outside of the critical radar and has a modest impact, it will likely have modest resources to be cost-effective. This way, the excess of requests can impact the service, deteriorating its response time or even making it unavailable, which would likely cascade to the upstream services.

The load peak can even occur due to other operations happening in the architecture. For example, imagine the taxes service needed to update all the system's taxes and temporarily request information from the location service. The bulk operation and the added load might impact the location service in the same way, cascading the failure to the critical order fulfillment flow. A complex network of synchronous requests is often very susceptible to these kinds of issues. When an apparently innocuous operation in one end of the system ends up impacting the other end, or the whole system, it might make us wonder if a distributed microservice architecture is any better than a monolithic application. In fact, when we suffer from these kinds of challenges, we are closer to a distributed monolith than a real decoupled and horizontally scalable architecture.

Event-driven architectures are still susceptible to these kinds of issues; synchronous requests are often a reality in most architectures. However, an event-driven approach offers a way to mitigate the impact of cascading failures naturally. How would the previous failure affect an event-driven architecture? Figure [7-3](#) illustrates the same use case.

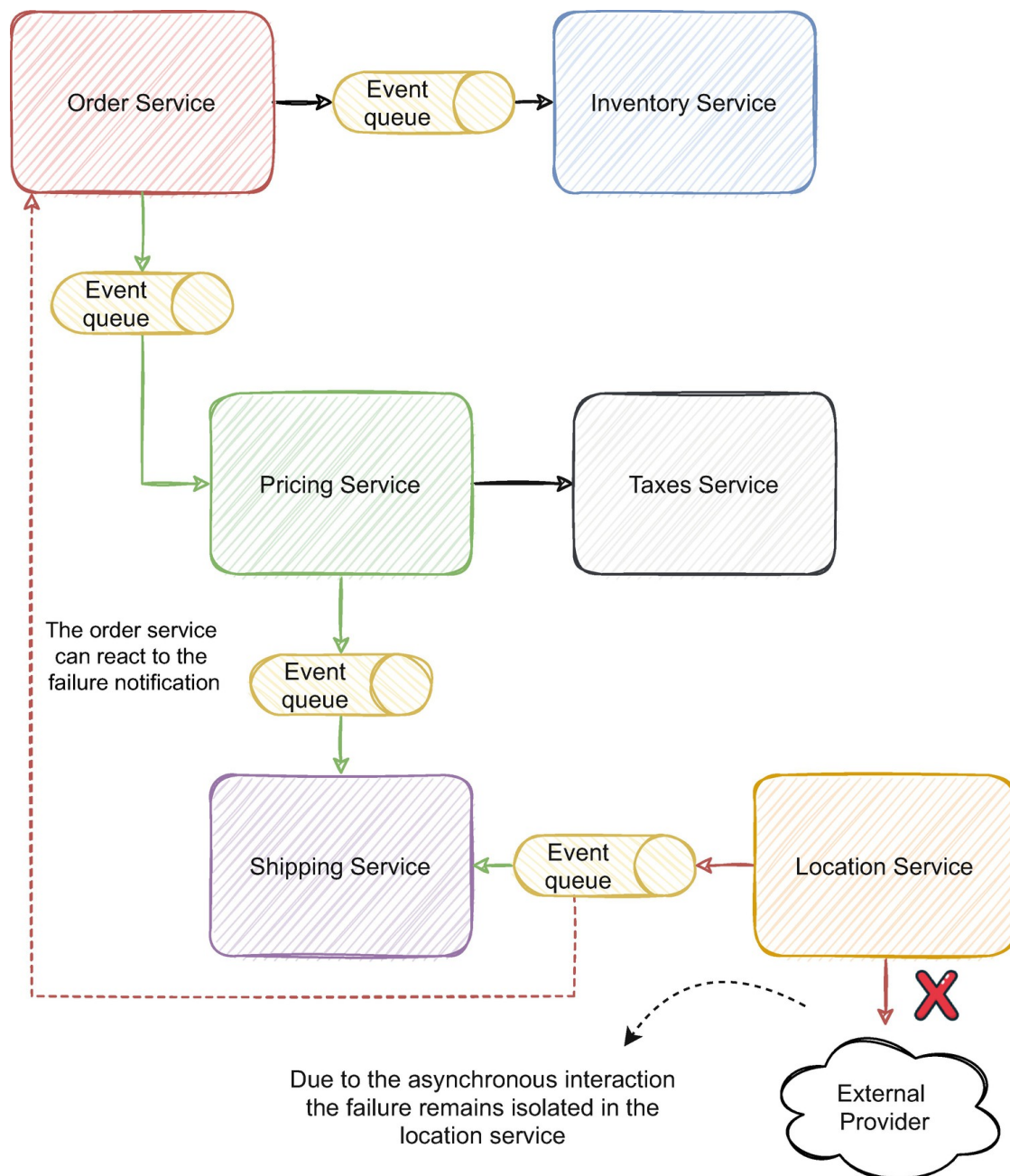


Figure 7-3 The same failure in the external provider remains isolated in the location service due to the decoupling provided by the event queues

The failure in the external provider and the requests by the location service remain isolated in that service. The decoupling provided by the event queue mitigates the cascading effect of the failure. Let's say the shipping service used the location service to retrieve additional information about the geographical details of each country. Using the event queue and building an internal state with that information (in a similar way that we discussed in Section [4.6](#) about building multiple read models), the service becomes autonomous and avoids the direct dependency. There is still an impact; new changes to those geographical locations aren't updated while the location service is unavailable since no new events reach the

shipping service. However, the impact is significantly smaller than stopping the whole order fulfillment flow.

7.1.2 Load Balancing and Rate Limiters in Event-Driven Services

The previous subsection mentioned how additional load to synchronous microservices could trigger cascading failures. A strategy to typically manage additional load and load peaks is the use of load balancers. Rate limiters can also be a valuable strategy to guarantee the service keeps responding to some requests while denying others. This subsection will detail this concept in a typical microservice architecture and how it relates to event-driven architectures.

As we discussed, a service can fail due to a myriad of reasons. In production, we should always have more than one instance per service to guarantee high availability. If one instance fails, becomes unresponsive, or has connectivity issues, the remaining ones can satisfy requests while the struggling instance is recovering. Microservice architectures typically accomplish this with load balancers. Load balancers keep track of the responsiveness of the service and route traffic accordingly. If one instance has issues, the load balancer routes the unavailable instance's traffic to the other healthy instances. Figure [7-4](#) illustrates the order service requesting information from the inventory service through a load balancer.

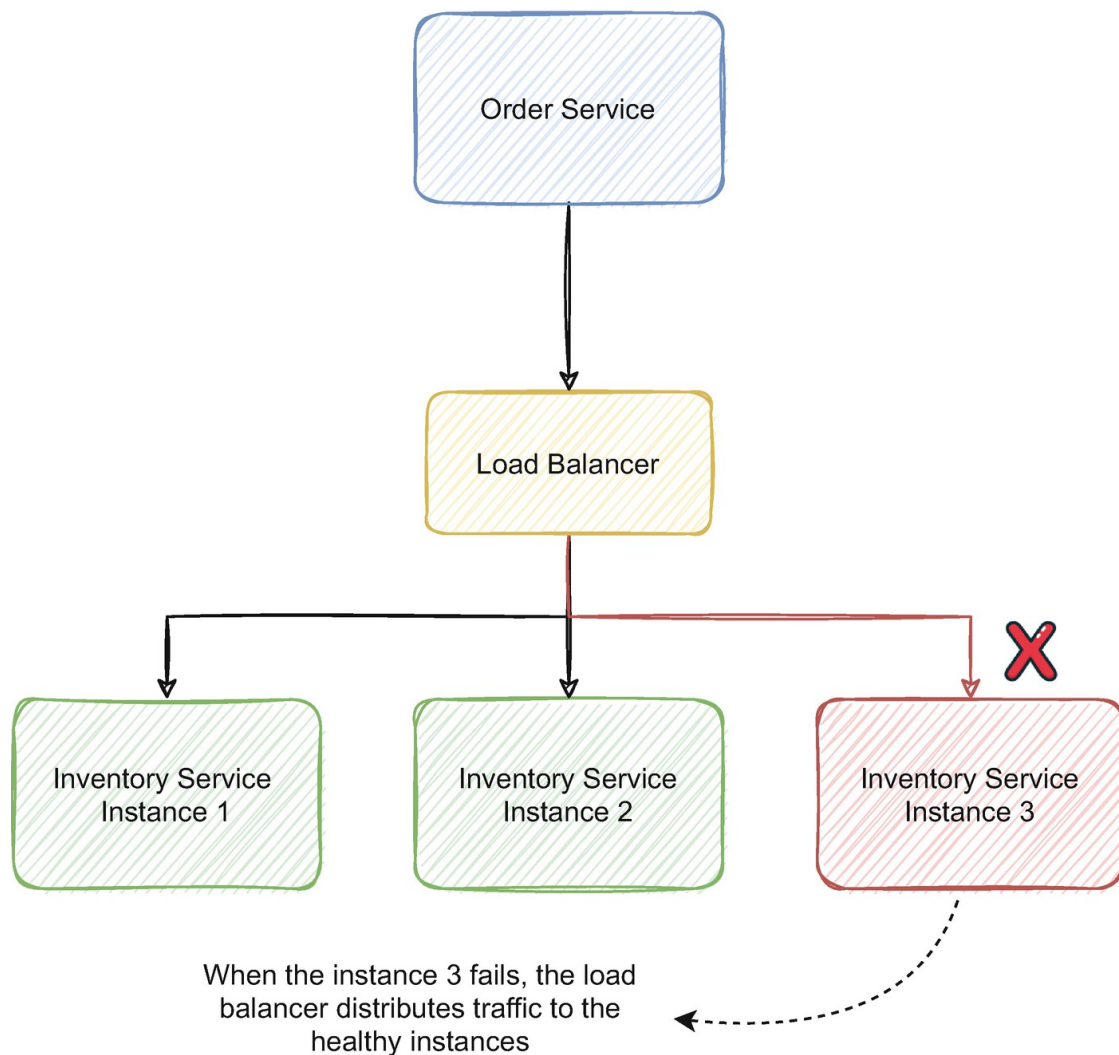


Figure 7-4 The order service requests stock information through a load balancer

When the inventory service instance 3 becomes unavailable, the load balancer routes the instance's traffic to the other healthy instances. Having three instances of the service provides higher availability. For the inventory service to be completely unavailable, all three instances would have to fail simultaneously, which is much more unlikely than one instance failing. Load balancers can also understand if the service response time is degrading instead of only reacting if the instance is unavailable.

Load balancers are usually relevant to instances that use synchronous requests. In event-driven services, the load balancing is generally managed by the event broker. The broker knows each instance's state and distributes messages to each one, as illustrated in Figure [7-5](#).

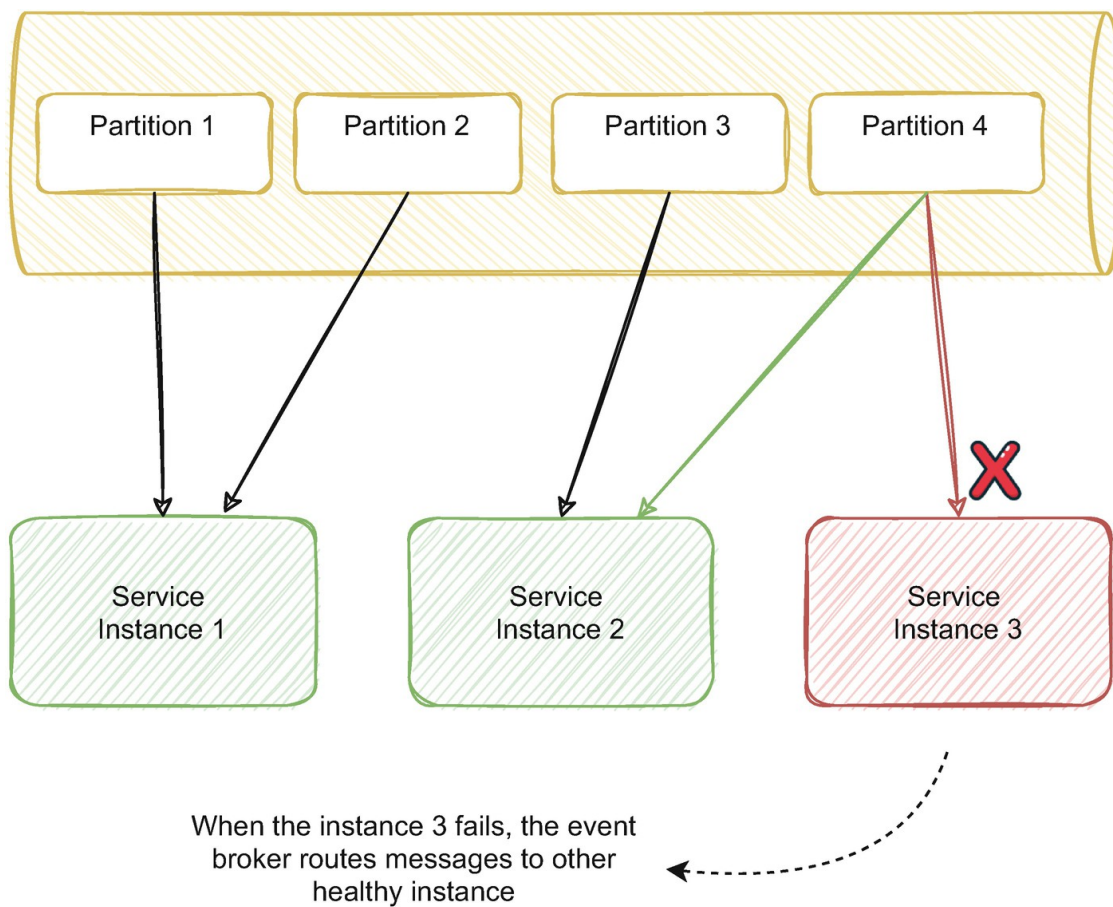


Figure 7-5 Load balancing through a message broker

You might recall when we detailed the anatomy of a Kafka topic in Chapter 6, a topic in the message broker can be divided into several partitions, and each partition is assigned to one consumer. In this case, instance 3 becomes unavailable; the broker can detect that the instance is down and reassigns the partition to another instance. The event broker can act as a load balancer without an external tool to manage it; the event consumption enables the balancing to occur more naturally. If it was a temporary failure, for example, the service loses connection to the broker temporarily, the broker will reassign a partition to the recovered instance when the service becomes available again.

A common reason for service failures is unexpected load peaks or large amounts of requests. DDoS (distributed denial of service attack) is often accounted for by the exterior architecture's security layers against external attacks. However, a similar common issue is excessive requests by internal services, a sort of internal DDoS attack but unintentional. Typical microservices often establish and enforce rate limits to guarantee the reliability of the service.

However, rate limiters typically imply rejecting some requests; when they surpass a specified threshold, the service denies a part of the requests. There are different strategies the service can apply besides blindly refusing a portion of the requests. It can prioritize critical business traffic or prioritize pivotal clients. However, it implies failing a segment of the requests and informing clients the request was refused (through an HTTP status code 429 – too many requests, for example). Upon receiving this response, the clients should adapt and moderate the number of requests. This strategy is known as backpressure and is applicable depending on the use case; rejecting requests is often (or always) not ideal. Applying backpressure also pushes the complexity to the clients; they need the rejected requests, so it often forces clients to use ad hoc retry-later strategies.

These kinds of approaches often seem like a workaround; we have a synchronous process that can't handle the required throughput, so we refuse requests, which forces the clients to apply a semi-synchronous approach by retrying the request later. Fully embracing an asynchronous approach is often more resilient and transparent. Rate limiters are arguably irrelevant in event-driven systems. If the service is receiving a higher throughput than it can handle, the impact is to build up lag (an increase in the number of messages to process) rather than failing. The broker naturally queues higher throughput than the service can handle. The queue piles up events to be processed later rather than pushing that responsibility to the clients. This strategy is far more resilient and avoids rejecting or failing requests. The services' horizontal scalability also provides the opportunity to increase or decrease the number of instances (even automatically) to guarantee that the lag doesn't grow to troublesome values since it can impact the eventual consistency, as discussed in [Chapter 5](#).

The services that expose synchronous requests benefit from having a deliberate load balancing and rate limit strategy in place. However, for fully event-driven services, load balancing and rate limits are naturally imbued in how event streams work and don't need an ad hoc or external dependency to manage it, unlike typical microservices. This characteristic

enables the system to be more resilient and adapt to varying throughput needs or unexpected unavailability situations.

7.2 Understanding Message Delivery Semantics

Until now, we described the interactions with the event broker as atomic and one-time operations. However, as you might imagine, it's not that simple. As we discussed in the previous section, services and other components can fail, and it can impact the overall system's availability and the reliability of the message communication. Services, and the broker itself, can fail and become unavailable while publishing and consuming messages. This section will detail how these failures can affect message consumption and production.

Let's work through a simple publish/subscribe example. Figure [7-6](#) illustrates a common use case of the examples we discussed in the previous chapters; the order service publishes an event to the message broker and the inventory service consumes it.

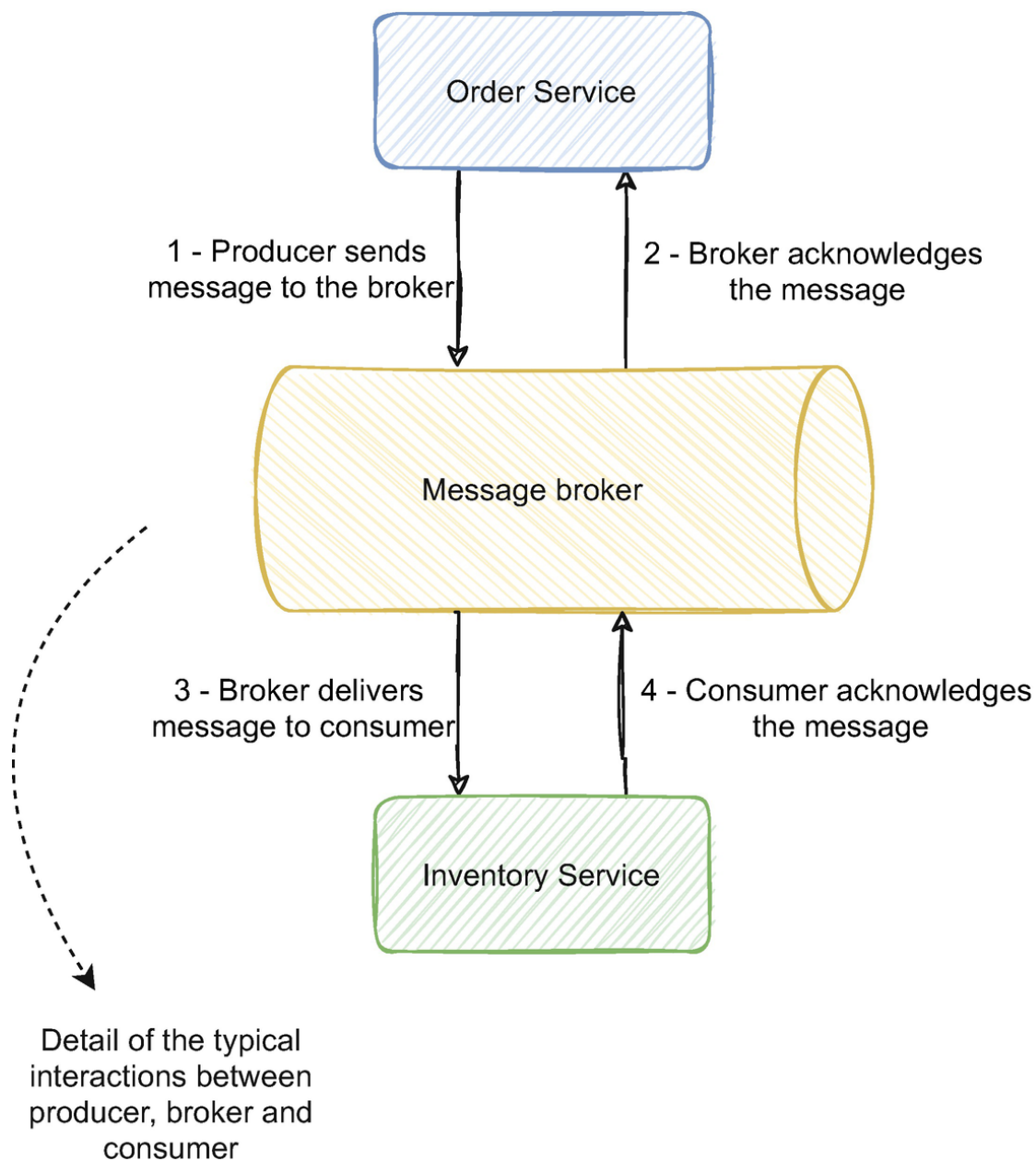


Figure 7-6 Detail of the interactions with the message broker

As depicted in the example, when the order service publishes an event, there are usually two steps associated with it: the transmission of the message from the order service to the broker and the broker's acknowledgment of the message reception. On the consumer side, it works similarly; the inventory service receives the message from the message broker and acknowledges it.

Each of these steps can fail and can originate different consistency issues:

1. The broker, or the connection between the order service and the broker, can be unavailable. The order service will fail to publish the message to the broker.
- 2.

After receiving the message, the broker can lose the connection with the order service, or the service can crash, failing to acknowledge the message's reception.

3. The inventory service, or the connection between the inventory service and the broker, can be unavailable. The broker will fail to deliver the message to the consumer.
4. The consumer, in this case the inventory service, can fail to acknowledge the reception and processing of the message to the broker.

If the order service failed to deliver the message to the broker, it would likely retry the message since each of these failures can be, and often are, transient. Even on a prolonged outage, arriving requests would need to be retried sooner or later. Retrying step 1 wouldn't likely impact the stream consistency since no message arrived at the broker yet.

However, what if step 2 failed? The message arrived at the broker, the broker persisted it internally, but when it acknowledged the message, the order service crashed, or the network became temporarily unavailable. When the service recovered, it wouldn't have a way to know if the message publication failed or the acknowledgment failed. It would likely retry the message publication, but in this case, the message had already arrived at the broker; the order service just didn't know it had. The publication retry would generate a duplicate message on the broker.

A similar situation can happen on the consumer side. After receiving the message, the inventory service can crash or lose connection to the broker. If so, it has no way to confirm the message reception. In case of a crash, it would likely reprocess the last messages it was processing, leading to duplicate message consumption.

Depending on the behavior of the producer and the consumer, we can have different message delivery semantics:

- **At-most-once delivery:** Messages might fail to be delivered but won't be delivered more than once. For example, in Figure [7-6](#), if the order service publishes a message but the message broker has an internal error persisting the message and returns an error, the message might not have been processed by the broker. With at-most-once semantics,

the order service when faced with this error won't retry the operation. Meaning messages will never be duplicated, but not all messages will reach the consumers. We avoid duplicate messages, but we also accept that we can lose messages.

- **At-least-once delivery:** Messages will always be delivered but might be delivered more than once. In the same use case we discussed before, if the order service didn't receive an acknowledgment from the broker, it could retry the operation. The broker already received the message and only failed to acknowledge it. By retrying, we would end up publishing a duplicate message. Thus, we guarantee messages are always delivered but might be more than once.
- **Exactly-once delivery:** Messages will always be delivered to the consumers without duplicates. In this case, the order service might receive an error, and even by retrying the operation, the message would arrive only once to the inventory service. Exactly-once is the most desirable delivery semantic but is also exceptionally hard to guarantee in a distributed system.

Table [7-1](#) resumes the impacts of each delivery semantic.

Table 7-1 Recap of the characteristics of the different delivery semantics

Delivery Semantic	Pros	Cons
	No message duplication	
At-most-once	High performance Practical to implement	Might lose messages in failure scenarios
At-least-once	Guaranteed delivery Practical to implement	Might have message duplication in failure scenarios Performance hit

Delivery Semantic	Pros	Cons
Exactly-once	No message duplication Guaranteed delivery	Exceptionally hard to guarantee and implement

These kinds of problems are related to the two generals' problem.³ The challenges of coordinating several distributed components through an unreliable network make it exceptionally hard to achieve a strategy with strong consistency, like implementing exactly-once delivery semantics.

At-most-once delivery is often associated with higher performance due to the lack of verifications (waiting for timeouts, for example) or retries. However, it also means we might lose messages from time to time. For example, when deploying a new version of a service or application, the service might be processing messages; with this approach, you might lose some messages because of the processing interruption when the service or application stops for the deploy. Losing messages, more often than not, is not acceptable, especially in streams with business-critical data where history is important. However, it might be valuable to less critical and high-load use cases. For example, like telemetry data, if new recent data is published frequently and there is no value in the history, some use cases might afford losing messages in failure scenarios.

At-least-once delivery often has a performance hit, but it usually isn't very significant; after all, failures happen but don't happen that often. It states that messages will always be delivered but might have duplicates. Duplicate messages, depending on the implementation, can generate erroneous results or duplicate work.

Exactly-once delivery is exceptionally hard to guarantee in a distributed system. Even if we only focus on the interaction between the producer and the message broker, in order to guarantee exactly-once delivery, we likely need the coordination between our services and the mes-

sage broker. Not all message brokers expose this functionality; some do, as we will discuss next with Kafka. Typically, end-to-end exactly-once semantics is only possible when all components in the flow participate in the coordination. Even with brokers that enable it, they usually do so in the message production and consumption. In use cases where consumers depend on external dependencies to process the event (like a database or an external service), it is extraordinarily hard to extend these guarantees to those dependencies. Exactly-once delivery provides the ideal guarantees but is often impractical.

Where does that leave us then? Two out of the three delivery semantics might leave us with incorrect or missing data, and the last one is exceptionally hard to achieve. It really depends on the use case, and we can't give you a one-fits-all rule, but here are some arguably practical pointers to help you decide. Exactly-once is excellent if the message broker supports it, doesn't have a considerable performance hit associated with it, and we don't need an overly complex solution to support it. At-most-once is suitable for use cases where you can afford to lose messages (data that is refreshed often, for example, the telemetry data we discussed). However, losing data is a terrifying thought and not viable in many scenarios.

If we rely on exactly-once semantics, we might implement event consumption with stronger assumptions that we likely need and make the system less flexible. For example, it might be harder to recover from a wrong state introduced by a bug (i.e., retries and event replays might become impracticable). At-least-once is arguably the most flexible and standard solution. Achieving at-least-once delivery can be straightforward, and by designing the event consumption to be idempotent (more on this in [Section 7.4](#)), we can avoid incorrect states if duplicates are found. We might have duplicate work, but it likely won't be significant if we process a handful of messages twice in a range of thousands or millions of events. Designing for idempotency makes the overall system more resilient and flexible (rewinding the event stream can be a valid approach, for example).

7.2.1 Exactly-Once Delivery Semantics in Kafka

Kafka introduced exactly-once semantics in the 0.11 release. It achieves exactly-once by implementing message idempotency with a sequence number similar to TCP and combining transactions across different topics. The throughput performance hit is arguably low, only 3% comparing with at-least-once and 20% comparing with at-most-once.

For systems that explicitly need exactly-once is a good alternative with arguably low impact. Implementing exactly once with Kafka is mostly configuration and straightforward.⁴ It has the caveat we discussed before when the consumption has the dependency of an external system; we need additional coordination to effectively achieve exactly-once semantics,⁵ which can be troublesome or impossible in some use cases.

However, as we discussed, we might benefit from designing for at-least-once and following ACID 2.0 (more on this in Section 7.4) to guarantee reliability and resilience. We imbue the message processing with additional flexibility, and we aren't tied down to more strict message processing guarantees.

7.3 Avoiding Inconsistencies When Saving State and Publishing Events in Event-Driven Microservices

Event-driven services usually react to events and apply their own domain logic, usually generating a different state in the domain's entities. They often need to save state and publish an event reflecting that change. We discussed the importance of the event stream's consistency and how we use it as a medium to share state across the organization. When we save state and publish an event, we are doing two different operations where any can fail. If one fails, we end up with an event stream that doesn't reflect the service's internal state. This section will discuss approaches to avoid this inconsistency, work through a use case, and discuss the possible solutions.

Most stateful services handle requests made to them by saving some sort of state. For example, Figure 7-7 illustrates the order service handling order submission requests.

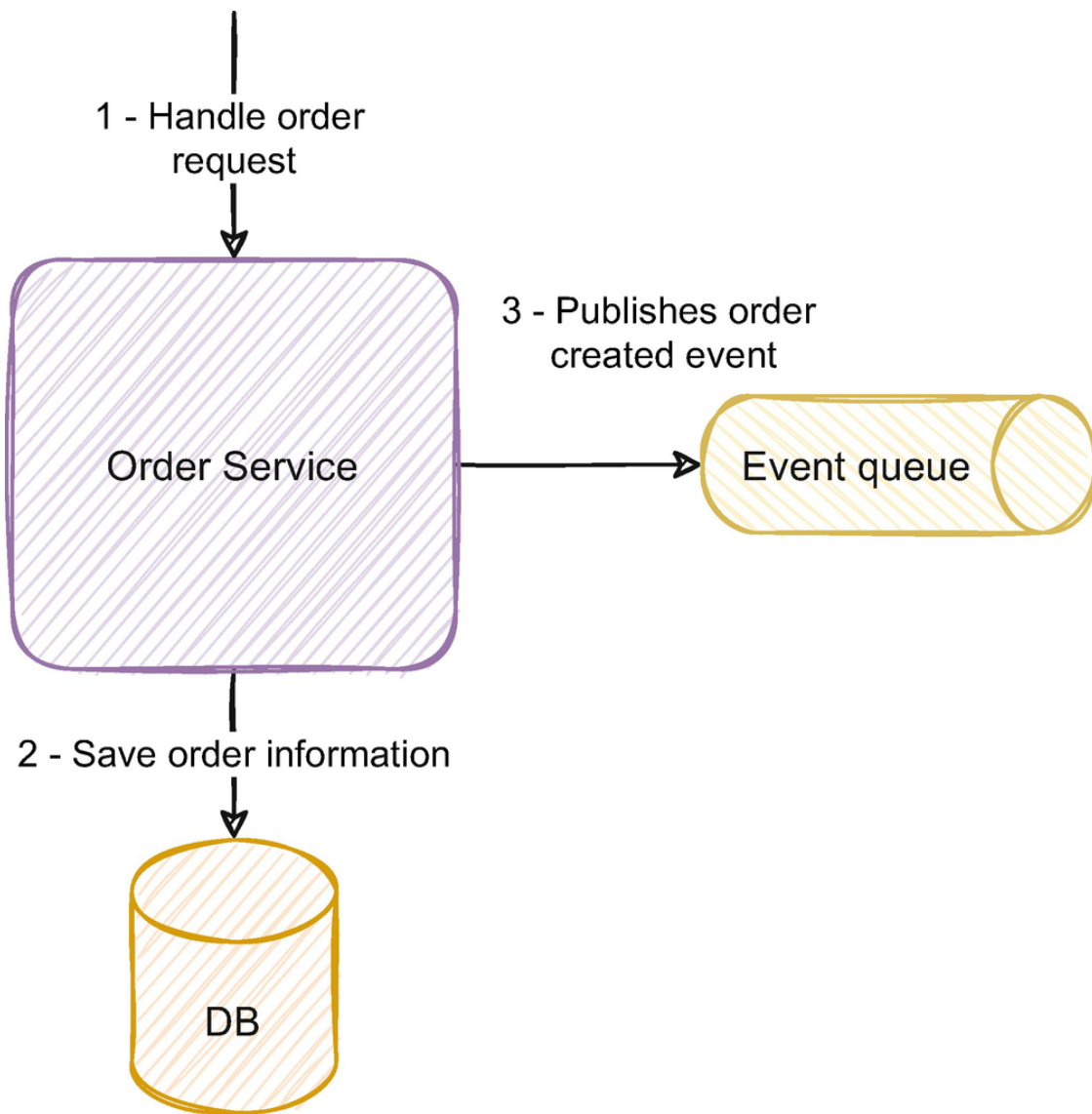


Figure 7-7 The order service handles order requests by saving state and publishing an event

The service saves the order information in the database and then publishes an event informing the ecosystem that an order was created. The challenge of having two steps in the order fulfillment process (saving state and publishing an event) is guaranteeing the atomicity of both operations. If the service faced an issue saving the information in the database, it would fail the request but wouldn't compromise the coherency between the database and the event stream. The failure is troublesome and needs to be addressed but wouldn't generate inconsistent states. However, if the service saved the new order but failed to publish an event, the event stream would diverge from the database. The stream would no longer represent the actual state of the orders and would be inconsistent.

This situation poses an additional complexity to the system's recovery. The service's connectivity to the message broker would need to be addressed, but that alone wouldn't be enough to return the system to a valid state. While the event broker, or the service's connection, was unavailable, the service already saved the order information to the database, but the service didn't publish the related events. Failing the request might not be admissible since the changes were already persisted. The event stream is the source of data for every other service depending on the order information. If the event stream is inconsistent, we propagate the issue to the whole ecosystem.

This issue is especially relevant for a choreography pattern we discussed in Chapter [4](#). Suppose the order fulfillment process depends on a series of choreographed steps. In that case, one service failing to inform it has finished processing the order, and without other mechanisms in place, it might stall the following operations.

7.3.1 Event Stream As the Only Source of Truth

As we discussed, event-driven also relates to the change of mindset in having a database as the source of truth and changing it to an event stream. In light of that, we could use the event stream as the primary source of information as opposed to the database. What does that mean to the example we discussed? We could publish the event and then update the current state by reacting to the same event. Figure [7-8](#) illustrates this situation.

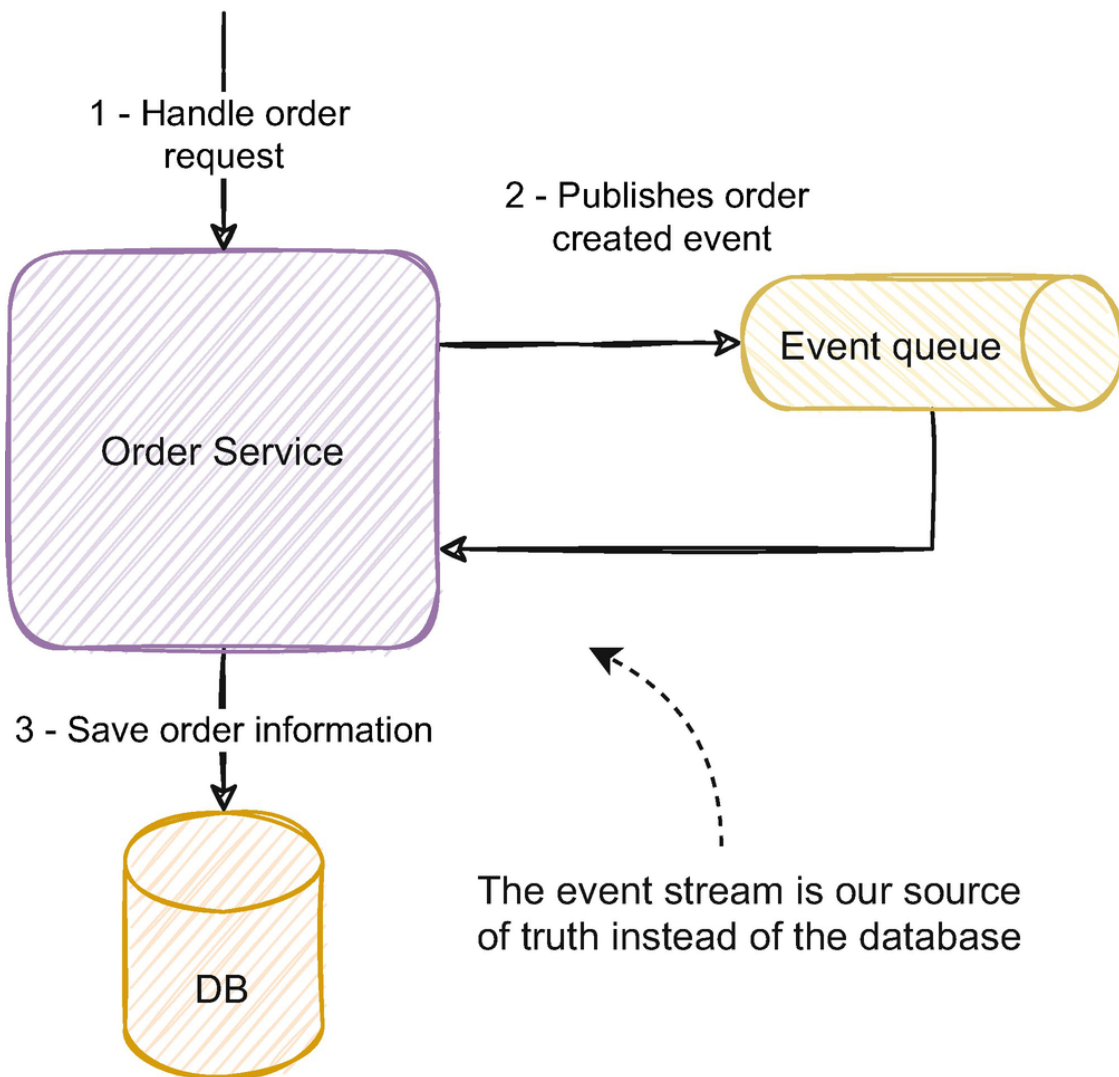


Figure 7-8 By saving state only when reacting to events, we turn the event stream into the source of truth

The order information is only saved by reacting to the order created event. This way, we guarantee the event stream is the source of truth; the current state is derived from the same stream. If the update to the database fails, we shouldn't acknowledge the processing of the event. Successful processing requires handling the event and updating the state (we will detail how to accomplish this in Section [7.5](#)).

This use case is very similar to the event sourcing and CQRS pattern we discussed in Chapter [4](#). The event queue effectively acts as an event sourcing store if the events are durably retained indefinitely. The event stream is also the system's write model, while the database is the read model. We could further segregate the responsibilities if we fully apply the CQRS pattern and detach writes and reads into separate services. Figure [7-9](#) illustrates this segregation.

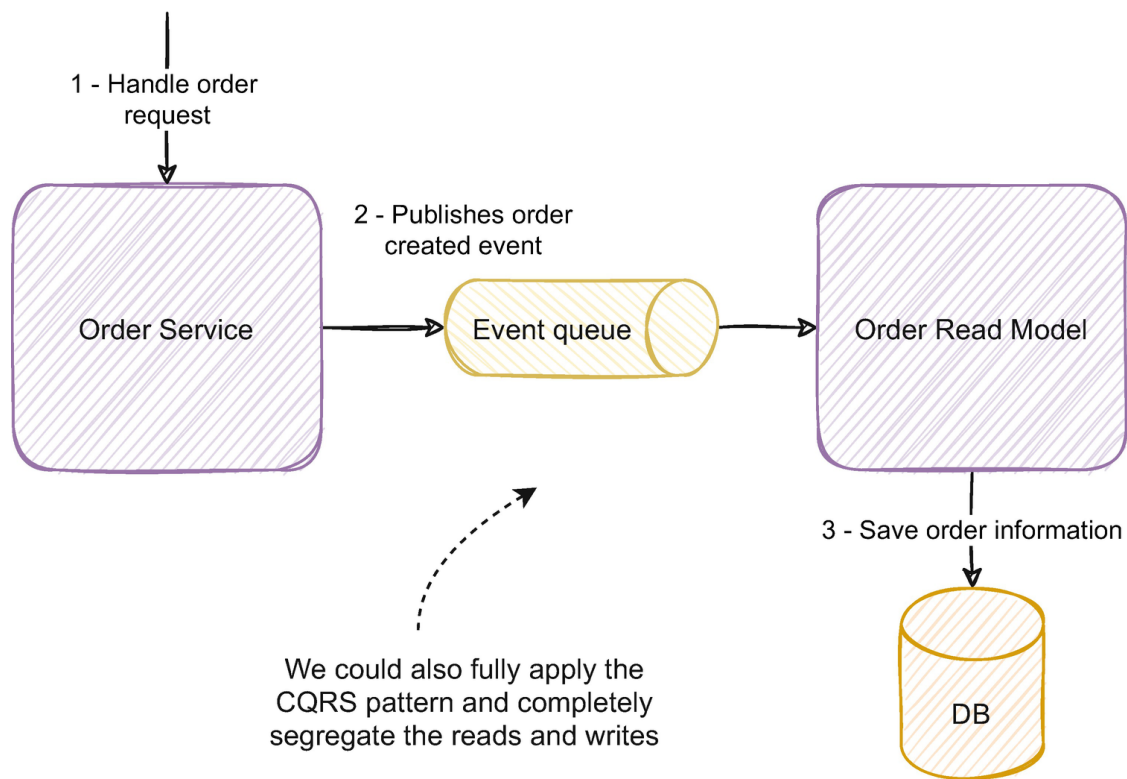


Figure 7-9 The same example but with fully segregated writes and reads

The order service receives orders and publishes events, while the order read model reacts to those events and updates each order's current state. The order service has the responsibility to publish events and the order read model to update the state in the database. This way, we can handle each failure separately and in a resilient way.

Although this solution aligns closely with the event-driven mindset, it has some associated caveats. This approach is a good solution as long as the service involved is a stateless stream processor, that is, it doesn't rely on a local state to apply its domain logic. If it does, it will depend on an eventual consistent store, and we risk performing domain validations against stale data. For example, in Figure 7-7, if the order service received a change to an order and needed to fetch the current order, there could be events in the queue that weren't yet applied against the read model.

Event sourcing typically loads all events from an aggregate (in this case, the order) and applies them sequentially, generating the latest state. However, most message brokers don't have this type of querying functionality. Unless we use the database as a queue, although not unheard of,⁶ other technologies are arguably more suited to do so. It often comes down to reading the whole queue and filtering the correct entity (or or-

der id) with durable message brokers. Although this is possible, it is highly impractical and resource-consuming for large quantities of events. It's often discussed whether Kafka, for example, can be used as an event sourcing store. There is an interesting article⁷ on how to achieve this with Kafka Streams and state stores; with it, we can avoid the challenge of consuming the whole topic.⁸ However, we might also run into the eventual consistency issue we mentioned. The services should apply domain logic and validations against the source of truth; if we can't retrieve the current entity's event stream, it can be troublesome to guarantee the correctness of the domain logic.

7.3.2 Outbox Pattern in Event-Driven Microservices

The outbox pattern is a standard solution when it's necessary to fetch state, apply domain logic, update the state first, and then publish an event. It typically involves two sets of updates, one for the entity's data and another for an outbox event table. The operation's atomicity is usually guaranteed with a transaction wrapping both operations, although we will also explore an alternative for non-transactional databases.

In the example we discussed involving the order service, let's say we save the order information in an order table. To apply this pattern, we would also create an outbox table that saves the events to be published. When receiving an order submission, the order service would insert the new order into the order table and the event into the outbox table. Figure [7-10](#) illustrates the two order service tables.

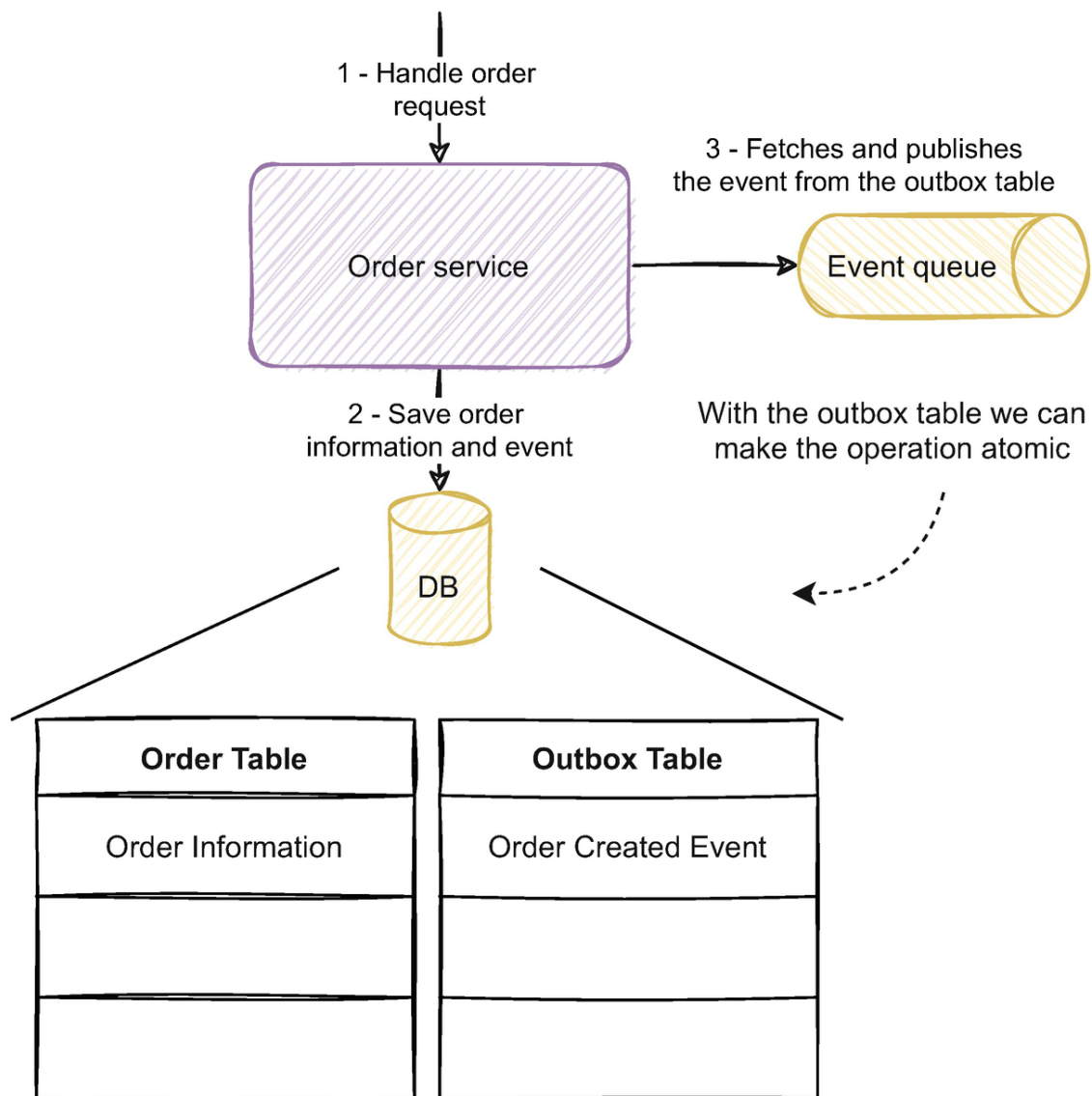


Figure 7-10 The outbox pattern uses an outbox table to make the operation atomic

When receiving an order submission request, the order service saves the order information in the order table and the event in the outbox table. Since both operations occur in the same database, we can use a transaction to make them atomic; they either fail or succeed together. An asynchronous process inside the order service can fetch events from the outbox table and publish them to the message broker. If the service saves the information in the database and fails to publish the event, it will still be saved in the outbox table. When the connection recovers, the process can fetch and publish events that failed. Once the events are published, the service can delete them from the table.

If the service crashed, there might be events in the outbox table that weren't published because the service might stop between saving the information in the database and publishing the events. When starting, the

service can fetch the events and publish them to the broker. To fetch the outbox table's events, we can implement an ad hoc process that polls the table. An arguably better solution is to do so only when a failure occurs, and the service starts. During regular operation, the service can save and publish directly without waiting for the polling process; this avoids delaying the process by the poll interval.

Instead of implementing an ad hoc polling process, we can use CDC (change data capture), as we discussed in Chapter [2](#). For example, Kafka supports this out of the box if we use a connector with debezium.⁹ It can detect changes in the database and publish the messages to Kafka directly.

This pattern guarantees the reliability of the process by relying on the ACID properties of transactional databases. However, what if we use a database that doesn't support transactions across different tables? A possible solution is to guarantee the database writes are idempotent (applying the operation several times always produces the same result; we will detail this in Section [7.4](#)) and not acknowledge the successful processing of the message until both writes succeed.

Let's say when saving the information in the order table, the operation is an upsert; that is, saving two times the same order produces the same result and doesn't fail due to a duplicate key error, for example. If the service saved the order information and then was unable to write to the outbox table (e.g., the service crashed), there would be an inconsistency; the service changed state but didn't publish an event reflecting that change. However, the service didn't acknowledge the command creating the order either. When the service recovered, it would process the same command again and write to both tables. Since the first operation is idempotent, it wouldn't effectively change anything, and the remaining operations would resume and publish the missing event. We will further detail these strategies in Sections [7.4](#) and [7.5](#).

7.3.3 Transactions and Compensating Actions to Avoid Inconsistencies in Event-Driven Microservices

The first option that likely comes to mind to solve this kind of consistency issue is to use transactions. If we are using a database that supports them, one possible solution to avoid this problem is to wrap the whole operation in a transaction. The service would open a transaction, write to the database (the save state step we discussed in the beginning of the section), publish the event, and commit the transaction. If the publishing fails, so will the transaction. However, we can't forget the database and the event stream are two separate components. An important detail is that we can publish the event, and then the transaction can fail to commit. The connection to the database can fail transiently or be unavailable after publishing. In that situation, we would have inconsistency, but the other way around, the event stream could have changes that were persisted by the service.

When the event publishing fails, instead of using transactions, we could undo the changes that were persisted in the database. We could have a series of compensating actions that reverted what was done before the event publishing. Compensating actions are also applicable to databases that don't support transactions. However, they suffer from a similar limitation; if the compensating actions fail, we end up with an inconsistent state.

Although still having a chance to fail, transactional and compensating actions significantly reduce the likelihood of leaving an inconsistent state. Saving state without publishing an event and vice versa only happens if the connectivity to both the database and the message broker fails at the same time. In [Chapter 6](#), we discussed how unlikely events are almost inevitable with enough scale. If that scenario can happen, then it will certainly happen eventually.

However, we should also have a pragmatic view of problems and solutions. If the chance of an issue occurring is remote enough, it might not be worth the effort of a flawless solution; we might be able to live with some issues if their impact isn't relevant enough. This kind of reasoning sends a chill upon my spine, and if you dealt with production systems for long enough, it probably sends a chill upon your spine too. I wouldn't advise following this line of reasoning as a standard, obviously, but we also

need to be pragmatic and choose the right tradeoffs. Transactions and compensating actions might be a viable solution in that scope; they are often familiar and straightforward to implement when a more sustainable approach is impractical.

7.4 Applying ACID 2.0 As a Resilience Strategy in Event-Driven Microservices

Failure in distributed systems is given for granted as a variable we must incorporate and account for in our solutions. Typical resilience strategies incorporate retrying strategies in some way. Retrying techniques are also applicable in event-driven services; when processing a message if a service's dependency or the service itself fails, a common resilience strategy is to retry the message consumption. Consuming a message that was already consumed in the past can be troublesome depending on the service design. This section will discuss how to apply the concepts of ACID 2.0 (associativity, commutativity, idempotence, and distributed) in event processing and use them to achieve fault tolerance.

Consuming repeated messages can become a common use case, for example, if we need to replay the event stream to repair data that was corrupted due to a bug. Retrying is fairly helpful when transient failures occur; in fact, transient failures are typically more common than outages that span a considerable amount of time. However, if the proper measures aren't in place, doing the same operation more than once can generate erroneous results.

Imagine if we have a service that manages the product's inventory and publishes events for stock changes. The product service stores information of all the product catalog, including stock. To update the stock information, it handles the stock events published by the inventory service. Figure [7-11](#) illustrates this example.

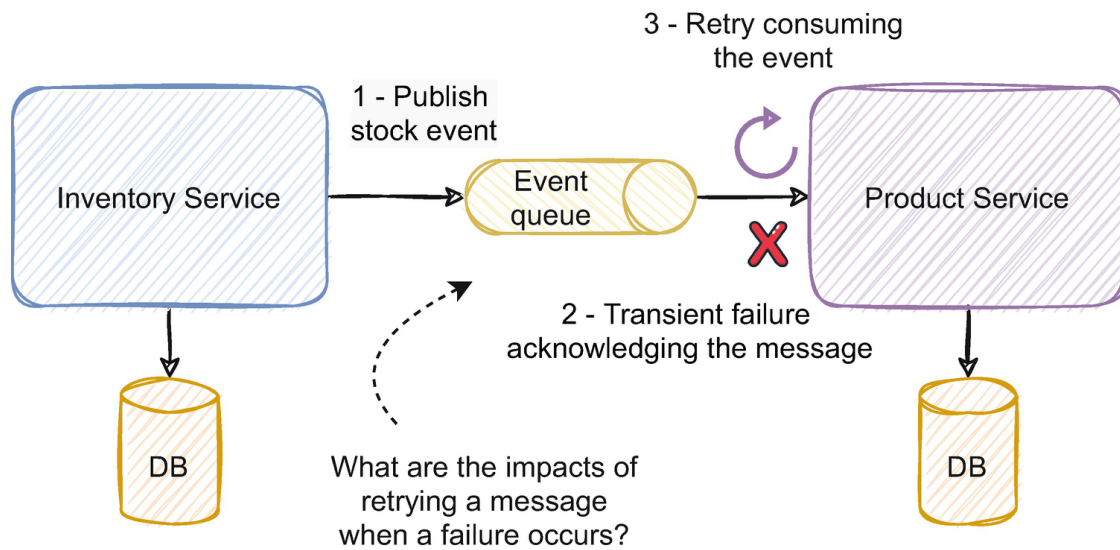


Figure 7-11 The product service retrying a stock event after a transient failure acknowledging the message

The example illustrates the product service facing a transient error acknowledging the event processing when consuming an event from the inventory service. On the first attempt, the product service can retrieve the event and write the changes to the database but fails to acknowledge the successful processing to the event broker. As a resilience strategy, the product service retries to consume the same event. The retry process will consume the same event and process it from the beginning. On the second attempt, the service can acknowledge and successfully process the event. However, the service wrote the stock changes to the database twice, producing a wrong value for the product's stock depending on how the product service consumes the event. For example, if the inventory service published a stock out event signaling the product stock was decreased, it would subtract the stock twice.

As we discussed in Section [7.2](#), exactly-once semantics is often complex to guarantee. To assure every message is processed, we often rely on at-least-once message delivery semantics. A service may receive repeated messages due to the message broker's failures or the upstream services, thus the at-least-once semantic. Under those circumstances, services should be able to receive and generate valid states with duplicated messages.

Event processing often benefits from understanding a set of principles about maintaining distributed consistency captured by the CALM¹⁰ theo-

rem. CALM (consistency as logical monotonicity) states that systems which process a progressive set of facts without changing what already happened are safe to run on an eventually consistent store. This concept closely relates to the event-driven approach; events reflect the evolving state of the entities and represent a fact that already happened which shouldn't be changed. ACID 2.0 (associativity, commutativity, idempotence, and distributed¹¹) embodies a set of design patterns closely related to the CALM theorem. By applying these properties to distributed systems, we can achieve distributed consistency and logical monotonicity.¹²

Chapter 5 discussed the impacts of distributed systems in traditional consistency and the difficulties of maintaining regular atomic consistency under the CAP theorem. The clever acronym behind ACID 2.0 raises the challenge of reflecting on a new kind of consistency under distributed systems, opposed to the traditional ACID in relational databases. ACID 2.0 is based on three main properties, associativity, commutativity, and idempotence, which happen in a distributed system (the D in ACID 2.0).

Associativity and commutativity translate the system's ability to perform out-of-order operations, which translates to the ability to tolerate unordered events in an event-driven system. Idempotence is the ability of a system to perform the same operation multiple times and obtain the same outcome. In event-driven systems, it translates to the ability to process the same event multiple times and obtain the same result.

How can we apply the ACID 2.0 properties to event processing? One possible approach is related to the event schema and how events are designed. In the example we discussed before, the stock out event (an event signaling the user removed or bought stock; basically the stock for this item was decreased by a quantity) isn't idempotent. If we process the same stock out event multiple times, the service will decrease the stock by the number of times it processes it, producing a different result each time. Instead of publishing a stock out event, the inventory service could publish a stock changed event that informs the product's current stock quantity. Listing 7-1 illustrates an example of both events.

StockOut

```
{  
    ProductId: 15251212,  
    Size: "XS",  
    Quantity: 1
```

```
}  
StockChanged  
{  
    ProductId: 15251212,  
    Size: "XS",  
    CurrentQuantity: 5  
}
```

Listing 7-1 StockOut vs. StockChanged event

The events have similar information, but the stock out event represents a removal of stock quantity from an item. The stock changed event signals the current stock of the product. If the product service consumes the same stock changed event multiple times, it will always produce the same result; it will constantly update the product's stock to the same quantity. We changed the event schema to be idempotent. We also made the retry operation safe. However, we also changed the event's meaning; a stock changed event has typically less domain value than a stock out event. If the user is buying a stock unit and decreases the stock, the stock out event reflects clearly the intent of the user; a stock changed event doesn't translate it as plainly. Nevertheless, it is essential to adapt to our consumers' needs, and in this case, the stock changed event is arguably the most reasonable choice (we will further discuss event schema design in [Chapter 8](#)).

A critical consideration also related to the event schema is the fact that the stock out events are order agnostic. We just need to process them once; it doesn't matter in which order (not taking into account negative stock validations). If the service publishes a stock changed, the order matters; we only want the latest quantity, thus the newest stock changed event. We can manage the order with event versioning as we discussed in [Chapter 6](#). In this case, as with most decisions, it's about weighing the tradeoffs and choosing a set of properties above others. Order is often easy to manage; idempotency is more challenging and provides strong resilience properties, being more useful in similar situations.

However, changing the event schema isn't always possible. An alternative is to make the event consumption idempotent. In [Chapter 6](#), we discussed how to use event versioning to handle unordered messages. We can use a similar strategy to guarantee the processing is idempotent. For

example, if we only process each event version once, we can turn the event handling idempotent. In the stock out event example, if we store the versions we already processed, we can ignore it when receiving a repeated event. This way, consuming the same event multiple times has no impact. We can also achieve the same result by ignoring repeated changes by the id or the event's date, depending on the event.

When duplicated events can generate invalid results and we don't enjoy ordered, exactly-once guarantees, we may need to reason how we can turn the event consumption idempotent and order agnostic. Event schema or event versioning can be the solution; sometimes we can change the way we are consuming the events and make it idempotent; sometimes it can even be solved as a business process. One of the main takeaways we should interiorize is comprehending the impact of repeated events and establishing a strategy to make the consumption idempotent.

7.5 Avoiding Message Leak in Event-Driven Microservices

An inconspicuous detail we can often miss is how to guarantee messages aren't lost in the presence of errors. Message leak occurs when the service acknowledges an event as successful and then fails to process it due to an error; the service might lose that event without the correct approach. When processing messages, the standard way to retrieve messages from the message broker is to acknowledge them promptly. Although efficient and straightforward, when faced with unexpected failures, it can generate message leaks. This section will work through an example illustrating this situation and how we can address it.

You might recall when we detailed the interactions between the services and the message broker in Section [7.2](#) (Figure [7-6](#)) and discussed how the consumers acknowledge the event's processing after retrieving them from the broker. Figure [7-12](#) illustrates this situation with the pricing service.

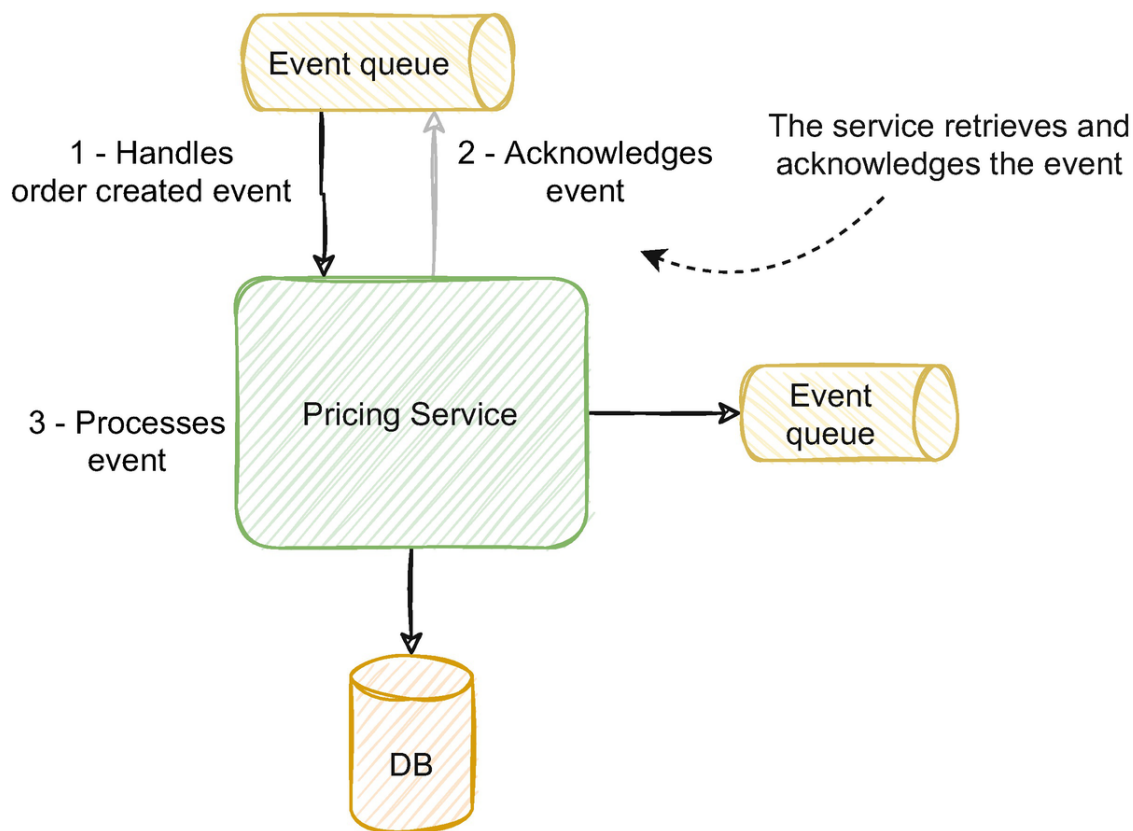


Figure 7-12 Example of an auto acknowledge interaction

The service retrieves an order created event from the message broker, acknowledges the event, and then proceeds to carry out the operations to process the event. This implementation is often the standard way to interact with message brokers; for example, Kafka has an auto-commit consumer configuration, which by default is enabled, which periodically and automatically commits offsets upon message reception. In RabbitMQ, you can choose between automatic and manual acknowledgments in the consumer (manual acknowledgments are the default).

This detail is essential because if the service crashes or one of the dependencies fails, we already acknowledged the message, the broker already marked it as consumed. In Kafka, the offset was already committed; in RabbitMQ, the broker already removed the message from the queue; etc. The service acknowledges the message to the broker, but it is still processing it in memory. Any issue that makes the service lose the message from memory will leak that message. This issue can happen if the services crash, or, depending on the deployment process, it may happen by deploying a new version. This means we might lose messages every time the service crashes or even when we deploy the service.

A more suitable approach is only to acknowledge the message after the processing is completed. The service retrieves the message; does all the processing it needs to, in this case, the pricing logic related to the order; and acknowledges the message. The service might have parallelism and process multiple messages but will only acknowledge each one after completing all operations involved in the event's processing. This way, if the service crashes, it will reprocess the events it was processing before it crashed instead of losing them.

Imagine the pricing service also publishes an event signaling the prices calculated for each order. These two approaches mark the difference between at-least-once and at-most-once semantics in the pricing service event publishing. If the service acknowledges the order event immediately, the service might crash before sending the pricing event but will never send repeated events. However, if the service acknowledges the order event after publishing the pricing event, it might send it twice. If the service crashes after the event publishing, it will reprocess the same event sending a duplicate event. Depending on the message delivery semantics we need, we can automatically or manually acknowledge messages. However, to not lose messages, acknowledgments should be manual.

7.5.1 Poison Events

There is a subtle difference between not losing events and ignoring events with corrupted or unexpected information. There might be events the service fails to process due to unforeseen data or failing validations. In those cases, should we ignore the event? If the service can't process the event due to failing validations, it might make sense to ignore it, but there can also be the case the service is failing to process events due to a bug or unexpected data.

We can fix the service and replay or change the offset to reprocess those events in those cases. Having a dead letter queue¹³ is also an option. When failing to process an event, we can publish it to a queue that only has failed events and resume regular work. Then an ad hoc process or a specific consumer can process or requeue the events. Uber has an article explaining how they implemented this with Kafka.¹⁴ Dead letter queues

might make sense when a large number of events have been published after the failing events since it might require reprocessing many events that already succeeded. However, with durable message brokers, since the events are persisted and still available, moving back the offset and reprocessing when possible is often a more simple solution.

7.6 Applying Common Resilience Patterns in Event-Driven Microservices

As we discussed at the beginning of the chapter, event-driven services might have a set of external dependencies. Each dependency or the network connection between the service and the dependency might fail. When a failure of this type occurs, there is a set of traditional approaches we can adopt. This section will detail these strategies and work through a practical example of applying them to event-driven services.

Event-driven services enjoy a higher level of decoupling and independence that avoids cascading errors to the upstream services. However, the service itself might face unexpected errors from its dependencies. The impact of those failures is often limited to the service's scope; for example, a read model might still respond to requests even though an upstream service is failing. Data will become stale since the service is no longer processing updates, but the event-driven architecture's nature allows the system to respond. It is often far more valuable to have the platform continue to respond even though a part of the system is unavailable.

Even though the scope is often smaller, the services have their own role and pivotal business value. The service dependencies might fail, and the service should be reliable enough to maintain correct operation and recover as soon as possible. We should also maximize availability and be resilient when facing a failure in a local dependency.

7.6.1 Retries As a Resilience Approach in Event-Driven Microservices

Retries are the standard approach to deal with transient errors. You likely implemented some kind of retry mechanism when accessing an external

service or dependency. In fact, the constant presence of transient and network errors highlights the usefulness of retries. A request might fail from time to time due to numerous reasons; retrying the request is often better than a manual process to recover data.

In event-driven services, we can apply a retry strategy in different operations of the processing flow. As illustrated in Figure 7-13, we can use a retry strategy while accessing external dependencies like other services or the database or applying a retry strategy to the event consumption itself.

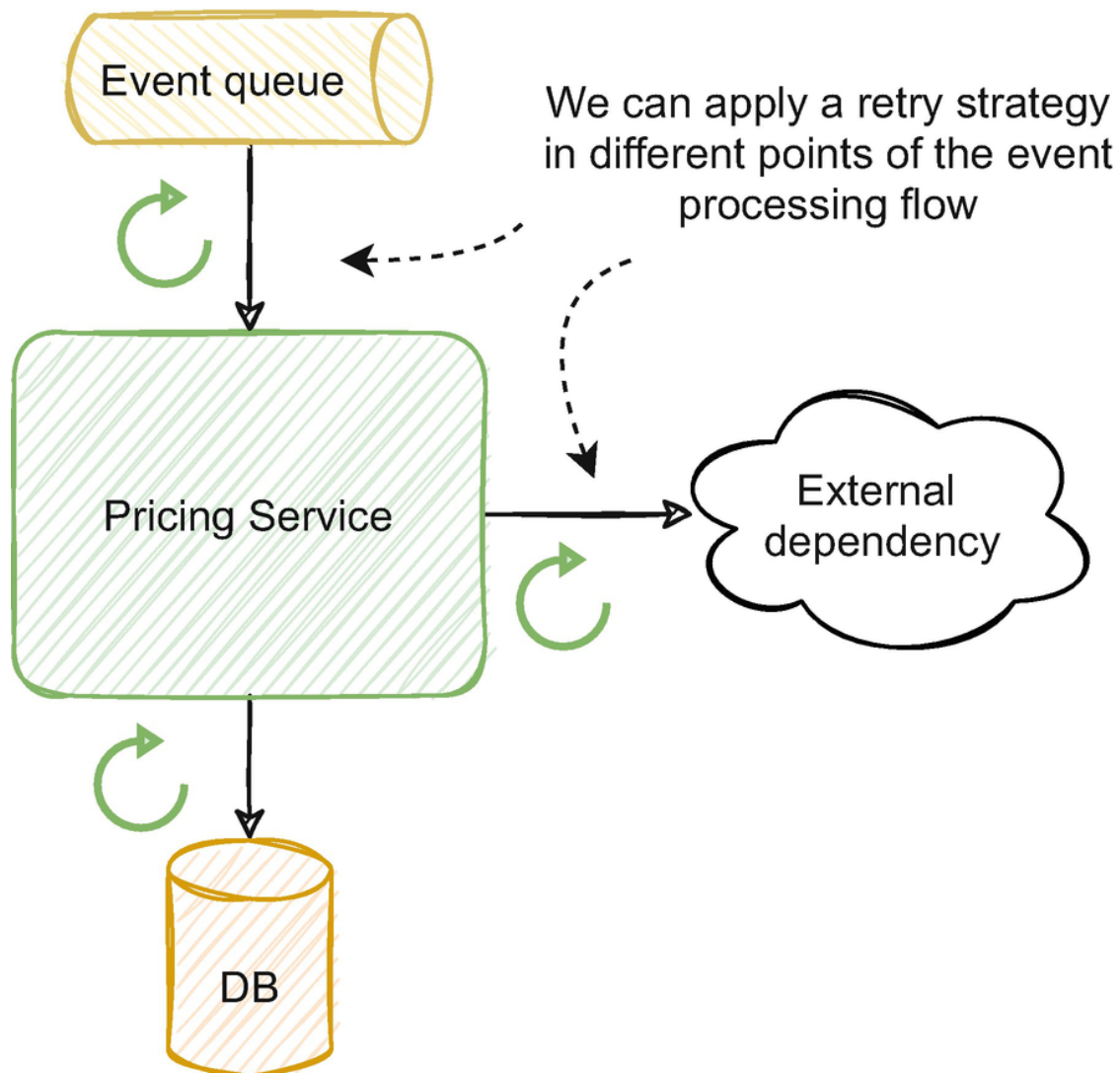


Figure 7-13 The retry strategy can be applied in different scopes

Applying retry strategies to external dependencies is often more useful when dealing with transient issues. We can also have retries at the event consumption level; when consuming, we can retry the whole event processing flow if an error occurs. This kind of retries is often more helpful when dealing with eventual consistency, optimistic concurrency, or in-

valid domain logic where we need to perform all the operations in the flow again.

There is a fascinating article¹⁵ detailing how Google achieved 99.9% latency SLA on some of their services by using retry strategies (we also mentioned this article in Chapter 5). Retries have higher resource consumption since the service has to make another request; however, as the article details, with the right approach, it can be significantly small (only 2% in the article use case) and significantly improve reliability and availability. Retries are often related to timeouts; when defining and triggering a timeout, we can retry the request. An arguably good approach discussed in the article is understanding the typical latency and specifying timeouts for the 95th-percentile expected latency before canceling and retrying the request.

As discussed with cascading failures, retries can be troublesome by adding additional load to an already struggling service. A common approach is to use retries with exponential backoff and a maximum limit. The retry strategy should always have a maximum number of retries to avoid more significant impacts or halting consumption. Imagine the service receives an event with unexpected data; if we indefinitely retry the consumption, the service will indefinitely try to process the event, impacting the throughput. Retries should also only be performed against conditions that are likely to succeed by repeating a request. We should be able to filter the conditions that will always fail.

Exponential backoff retries are also useful to limit the impact of the additional load on dependencies. We can successively make additional requests but separated by an exponentially increasing time interval between the requests. We can also increment a random value (also known as jitter) to randomize the time interval. The jitter helps to avoid accidental choreographed load peaks across different services.

We can also have different strategies when all retries fail. We might have dependencies critical for the service's logic or others that we might be able to continue processing, although with some degradation. For example, if the connection to the database fails, it might impact all types of events consumed by the service. However, if the caching mechanism fails,

we might still process events without cache, but perhaps with reduced throughput. When a critical component fails, an arguably good approach is to apply a circuit breaker (we will detail circuit breakers next) and not acknowledge the messages that failed (as discussed in Section [7.5](#)). This way, the service won't try to process additional messages and reprocess the messages that failed once the service recovers.

7.6.2 Circuit Breakers in Event-Driven Microservices

Retries are great for transient failures or short-timed unavailability. However, providing resilience while facing more extended outages (a failure that persists after several retries) might require different approaches. Retrying an operation that is unlikely to succeed might be pointless or aggravate an already difficult situation. Instead, the application should avoid frequent requests and handle the failure in the most suitable way possible. The circuit breaker pattern can be a safety net to prevent the service from making excessive requests. This subsection will detail how the pattern works and how we can apply it to event-driven services.

Event-driven services can have external dependencies, like the database or a distributed cache. For example, suppose we have a pricing service responsible for calculating the order's price according to country and discount rules. In that case, it could have a dependency on a distributed cache like Redis. Transient connection issues to Redis can be handled using retries, but we might benefit from applying a circuit breaker pattern to deal with longer outages. This example is illustrated in Figure [7-14](#).

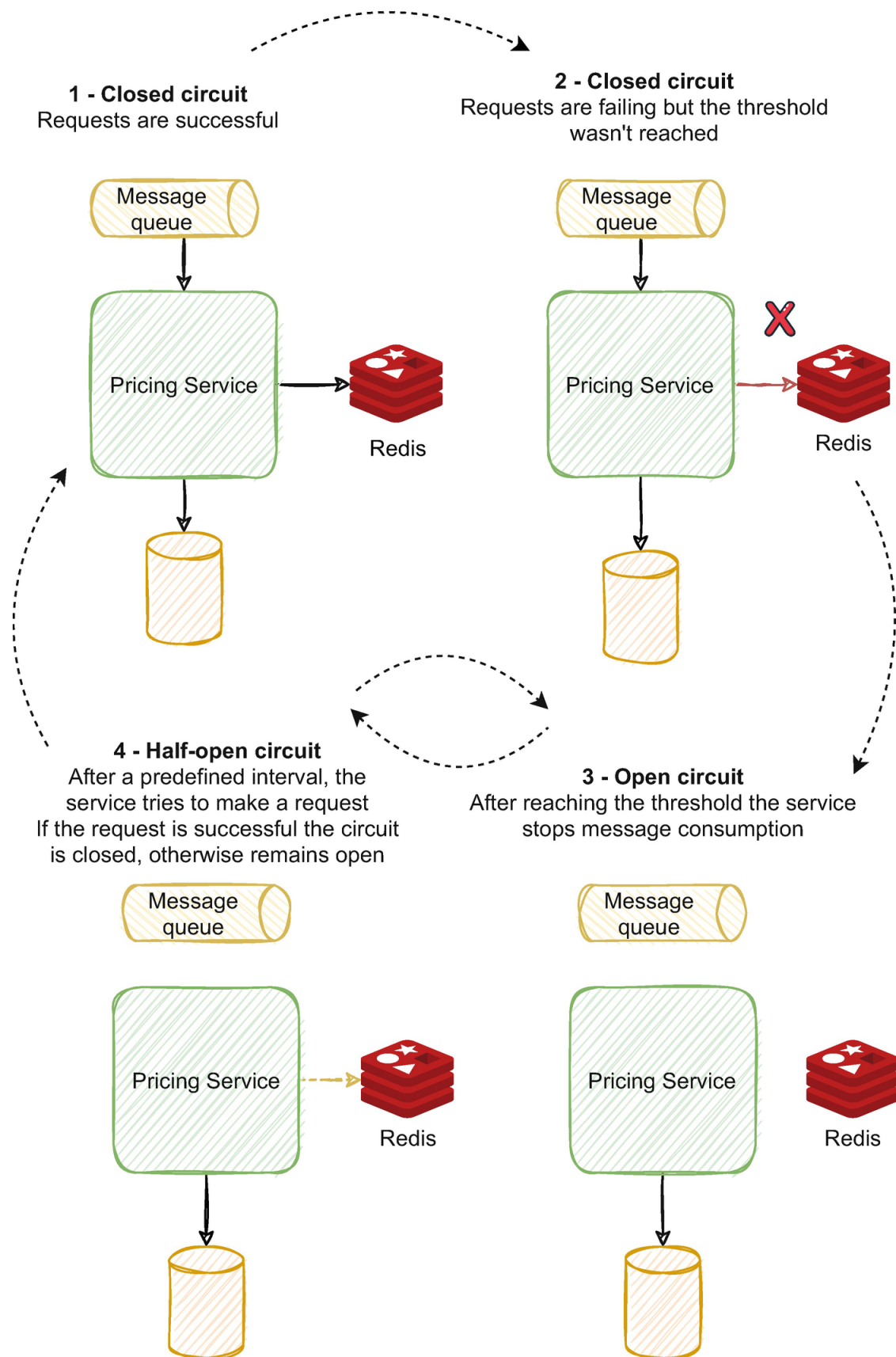


Figure 7-14 The different states of a circuit breaker pattern applied to an event-driven service

The pricing service reacts to order events and uses Redis and the database to process its domain logic. While in normal working mode, the circuit is considered closed; external requests are successful and events are processed. The concept of opened and closed circuits originates from ac-

tual electrical circuit breakers. They were electrical switches designed to interrupt current flow when faced with an issue; thus, when in a closed state, the current flows, and in an open state, the current stops. The pattern uses the same concept: when the service is processing normally, the circuit breaker is closed, and when faced with an issue, the circuit breaker opens and stops the processing flow.

If the pricing service has an issue reaching Redis, the requests will start to fail. When the number of failed requests goes above a defined threshold, the circuit breaker opens. In this case, it stops message consumption and consequently the requests to Redis. When in this state, the circuit breaker is considered opened. However, the service can't remain in this state forever; from time to time, the service will try to reach Redis. While the requests fail, the circuit will stay in the open state. When the requests finally succeed, it will close the circuit and resume message consumption.

One of the advantages of reacting to events in a queue is the service won't lose data or fail requests. The upstream services will continue to publish events; once the service recovers and resumes message consumption, it can process the events queued during the outage. The service can lag behind the latest changes happening in the ecosystem and might pile up work; however, it's better than failing or losing those requests. If the number of messages is significantly high, we can horizontally scale by adding more instances to reduce the lag faster and remove them when the lag clears.

Circuit breakers also open the opportunity to create a more resilient service by providing the possibility to choose a suitable fallback when applicable. In this case, instead of stopping event consumption, we can replace the external dependency with a fallback. For example, while facing a Redis outage, the service can fall back to an in-memory cache; it might not be as efficient as a distributed one but might be enough for the service to cope with regular demand while the outage recovers.

7.7 Recovering Data and Repairing State in Event-Driven Microservices

The previous sections discussed how to guarantee messages are sent and processed reliably. We also discussed how not to lose events and to ensure every event received is processed. With these approaches, we provide tools to control failures and avoid data repair processes. However, data can still be corrupted, or the services can generate an undesirable state (due to a bug, for example). This section will discuss how we can use durable event brokers to recover data and repair the service's state.

Let's work through a practical example similar to the ones we discussed in the previous sections. The pricing service handles order created events from a queue, processes the charges, generates an internal state in a database, and publishes an event informing the order's charges. We released new functionality that changes the way taxes are calculated, and, unfortunately, we introduced a bug. We quickly noticed the issue due to our comprehensive monitoring and alarmistic and rolled back the release. This situation is illustrated in Figure 7-15.

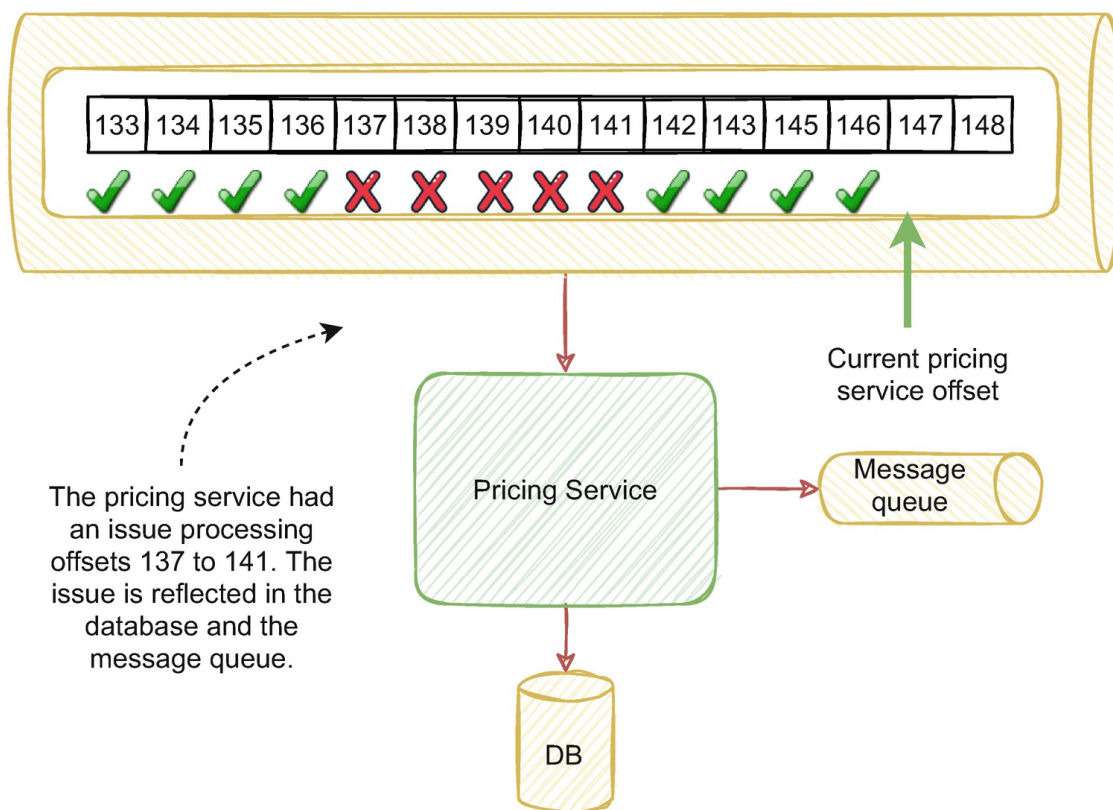


Figure 7-15 The pricing service generated invalid state while processing a set of events

However, the pricing service ran with an invalid version against a set of events. The service stored the state generated by those events, and it is safe to assume part of our data is corrupted. This situation has an additional aggravating factor; since the charges were wrong, the events the

service published are also impacted, which might also affect the downstream services.

In a traditional monolithic application, a possible solution would be to identify the affected orders and run an ad hoc process to recover them by running a database script. In that case, it might be somewhat straightforward since all data is in a single database, and we can correct everything in a single operation, even though it's often an error-prone process. In a synchronous microservice architecture, it might be more challenging; a database script wouldn't likely suffice since the pricing service's dependencies wouldn't be affected (in a synchronous architecture, the event queue would be replaced by a direct request to the service that needs the pricing data). The alternative could be running an ad hoc process to call the service to repair the affected orders. Identifying the orders with an invalid state can also be challenging if there aren't the querying capabilities to do so or there is an overwhelming number of orders.

In an event-driven service, an arguably straightforward solution is to change the service's offset to the offset before the bug was introduced. In the case of Figure [7-15](#), we could change the pricing service's offset from 147 to 137, and the service would reprocess the affected events (from 137 to 141), and the state would be repaired. The drawback is events from offset 142 to 146 would be reprocessed even though they were successful, hence the importance of idempotence we discussed in Section [7.4](#).

The service already published invalid events, and if we are not using documents, they should remain as is. As you might recall from Chapter [3](#), documents have the entity's complete state, and the event broker only retains the latest ones (as we discussed with Kafka compacted topics). It might seem odd, but events represent facts that happened in the past; if charges were effectively wrong, then the event history should reflect that fact. To repair the state, by going back in the offset, the service will publish new events with the correct information, thus repairing the downstream services that will react to those events. As we discussed before, the service only publishes an event if the state changed. In this case, the service would only publish events related to the invalid orders that the service corrected. Although the service reprocesses the already successful

orders, it won't change any state, reflecting in no new events, and thus won't affect the downstream services with additional load.

We mentioned the wrong events should remain as is, except for documents. In those use cases, the queue doesn't retain the history but rather the newest state. Since we would publish a new document with the correct information, the service would also correct the data in the document queue since the broker would only retain the latest documents. The invalid ones would be older and thus would be purged by the broker.

If we apply the other strategies we discussed in this chapter, this approach is likely limited to bugs and similar issues since outages are already accounted for. However, even if we don't have those strategies in place, this solution can be a valuable approach to recovering from leaked events or random outages.

Notice that this approach doesn't involve any ad hoc manual process or specific developments to recover the state. We use the same flow the service uses to process regular events; we just rewind the event queue and reprocess. The beauty of having a durable event queue, besides being able to derive value from history and generating new views of the data, is having all the tools needed to recover state sustainably.

7.8 Bulkhead Pattern in Event-Driven Microservices

Bulkheads are usually employed in ships as a means to contain failures. They separate the ship into several isolated partitions; if the hull takes damage and one section is compromised, the bulkheads guarantee the failure doesn't spread to the rest of the ship. A ship can still sail with some flooded partitions; the bulkheads avoid an otherwise catastrophic failure. This section will discuss how we can apply a similar concept to an event-driven architecture.

On a side note, the Titanic had bulkheads, but they didn't prevent the ship from sinking. Titanic's bulkheads didn't go full length due to mainly not hindering the passenger's comfort and movement throughout the

ship. Since they weren't tall enough, the bulkheads didn't contain the water in the damaged partitions, the software engineering equivalent of marking resilience developments as "nice to have" in planning meetings.

Anyhow, how do bulkheads apply to event-driven services? Let's use the same example with the pricing service. The service handles order created events, processes the charges for each order, and saves state in a database. Let's say we also have a taxes service in charge of managing changes in taxes across every country. We discussed why each service should have its own database; however, the database cluster can have multiple databases. For example, if we use MongoDB, we might want to group several domain-related databases in the same cluster. Perhaps more theoretical references might refer to this as an anti-pattern, and truth be told, the ideal situation is to have complete segregated database clusters for each service. However, this is often not the case due to practical and monetary reasons, and it isn't due to solid reasons. Although often oblivious to us, a solution that has a considerable cost to maintain, even though with excellent performance and scalability properties, isn't a very good solution. After all, we are building the tools for a business to strive.

When the taxes change, they produce a bulk operation in the taxes service, which is very database heavy. The additional load starves the cluster's resources and might impact both the taxes and pricing service, as illustrated in Figure [7-16](#).

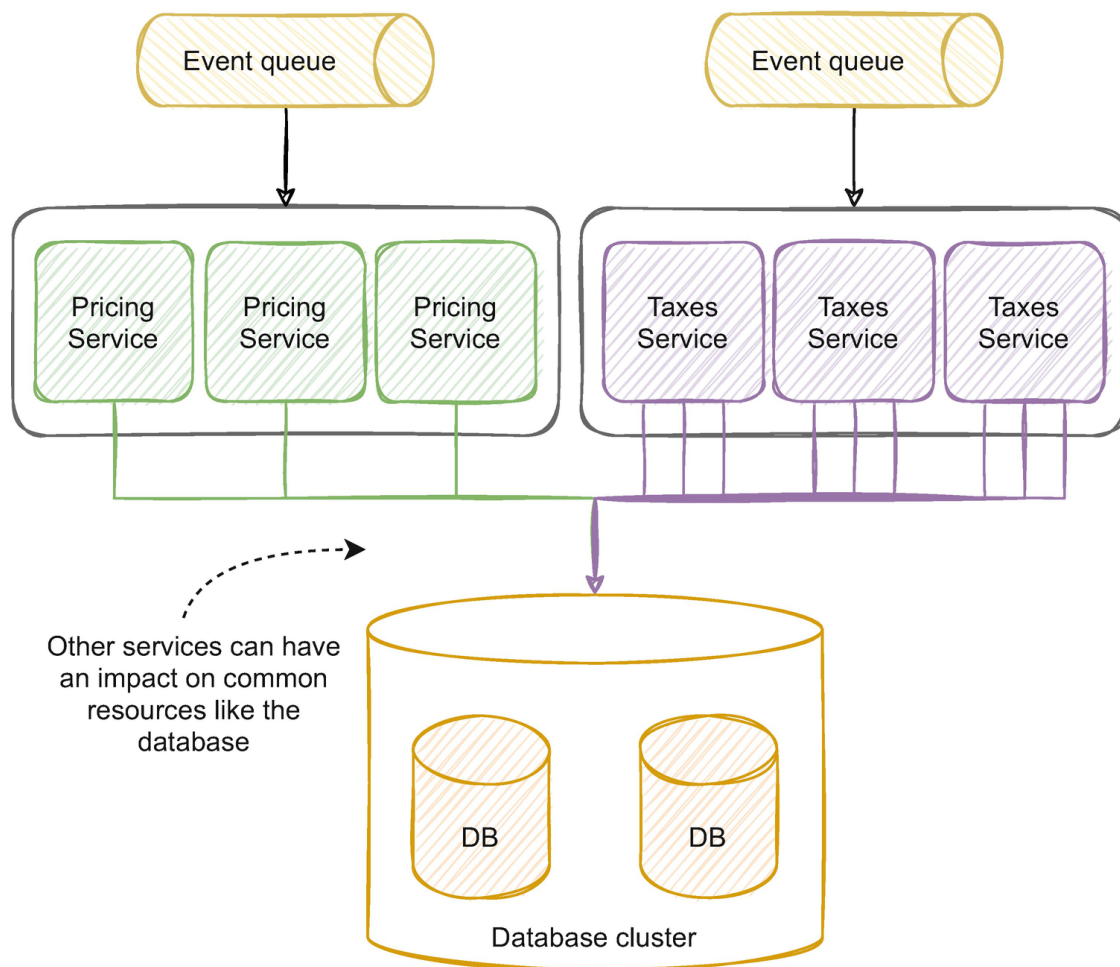


Figure 7-16 The taxes service produces additional load on the database cluster due to a bulk operation

In order to mitigate the impact done by the taxes service, we could apply the bulkhead pattern by adjusting the parallelism degree in the taxes service. We discussed in Chapter 6 we might have several instances of a service, and each service instance has parallelism to optimize physical resources. By decreasing the parallelism degree, we would also reduce the impact the service has on the database cluster. If the service has several instances, we can also reduce their number. The parallelism degree and the number of instances are essential parameters we should include in our solutions from the ground up to adjust the service performance and mitigate this kind of impact.

Obviously, the taxes service will process slower, but a bulk action might be more permissible to be processed slower than the charges applied to each order. The bulkhead pattern is about containing failure; the service might process slower, but it won't propagate the issue to the pricing service or other processes running in the taxes service. In this case, as a more sustainable solution, we could upscale the database or separate it

into different clusters in order to avoid the impact, as we discussed before. However, it might not justify the cost and might be more reasonable and cost-effective to say a bulk operation might take a while to finish with modest parallelism. We can apply this approach to any kind of dependency, a distributed cache, or a synchronous request to another service.

When using synchronous requests to other services, the bulkhead pattern might be an arguably good solution since those services might be susceptible to cascading failures. A bulkhead approach in the upstream services might quickly avoid widespread impacts.

7.8.1 Priority Queues

Another way to manage the impact of the unexpected load is to segregate operations into different queues. In the previous example, the taxes service received a bulk operation which might reflect an abnormal message arrival rate. Since, typically, a bulk operation has lower priority than the changes happening in real time in the ecosystem, we could delegate them to a lower priority queue.

By having segregated queues, we can allocate different resources to each queue. The queue with the regular everyday changes might have a higher degree of parallelism than the queue handling bulk operations. This way, we can adjust each queue's parallelism degree to guarantee there is no lag in the higher priority queues. The lower priority queues get processed at a sustainable rate without affecting the service's dependencies and resources. The assumption is the service might lag in the low priority queue, which should be alright since the changes don't need to be processed in real time. We can delegate longer-running processes to low priority queues and significant business changes to higher priority ones.

7.9 Conclusion

How all the patterns we discussed throughout the chapter are applied to the service's dependencies? Figure [7-17](#) illustrates how these patterns relate to the initial discussion of a typical service topology and its dependencies (Figure [7-1](#)).

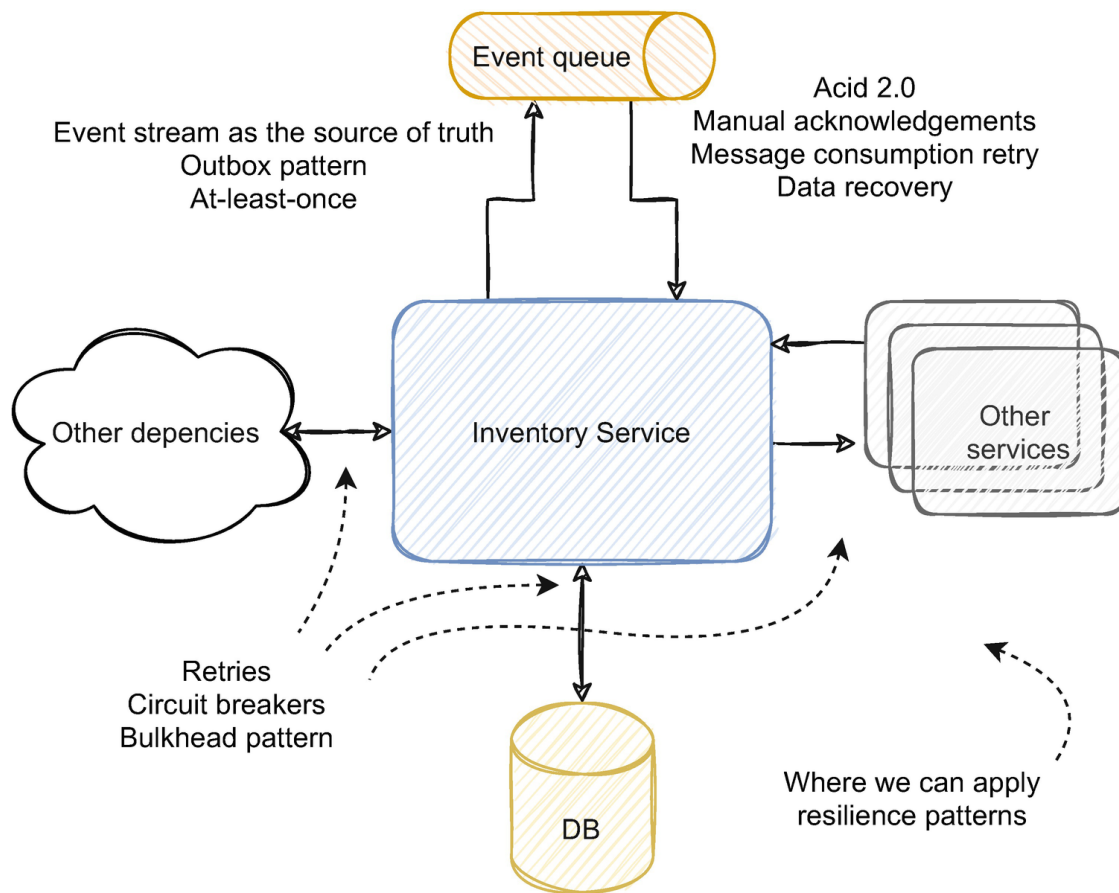


Figure 7-17 How the patterns discussed throughout the chapter relate to the service's dependencies

In typical external dependencies like other services, distributed caches, the database, etc., it's often relevant to use retries, circuit breakers, and the bulkhead pattern. When publishing messages, we can use the event stream as the source of truth, the outbox pattern, and publish in at-least-once semantics. We can consume messages according to ACID 2.0, do manual acknowledgments/commits, retry the message consumption, and use the data recovery strategy.

If we apply the patterns we discussed in this chapter, we can guarantee we process events reliably and without leaks. We also guarantee the safety of the service's dependencies and ways to regulate traffic and load.

7.10 Summary

- Services have several dependencies, and each one of them can fail due to a number of reasons. We need to guarantee these failures don't affect the reliability and the correctness of the service.
- Event-driven services are less likely to suffer from cascading failures due to their decoupled nature.

- Load balancing and rate limiting are often already incorporated in the design of event-driven services due to their interaction with the event broker.
- Event delivery can have three different semantics: at-least-once, at-most-once, and exactly-once. Exactly-once is the most useful and the hardest to guarantee. At-most-once loses messages but typically enjoys higher performance. At-least-once might generate repeated work, but it's straightforward to achieve and ensures no messages are lost.
- The service's state must reflect the event stream. We can achieve this by making the event stream the source of truth, with the outbox pattern, or using transactions and compensating actions.
- Applying ACID 2.0 properties to message consumption can provide higher resilience and flexibility, especially when associated with retrying strategies.
- Manual acknowledgments or commits can guarantee messages aren't lost in a failure scenario.
- Retries and circuit breakers are common resilience patterns we can apply to make the service more resilient and regulate throughput to external dependencies. We can combine circuit breakers with fallbacks to make the service resilient to common failures.
- Rewinding the event stream can be a powerful tool to recover data.
- The bulkhead pattern can contain the proliferation of local failures. Priority queues can also be a viable option to regulate traffic with different urgency without overwhelming the service's resources.

Footnotes

¹ Further details in “Fallacies of distributed computing,”

https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing.

² Full presentation by Yuri Shkuro, September 9, 2019, “Conquering Microservices Complexity @Uber with Distributed Tracing,” www.infoq.com/presentations/uber-microservices-distributed-tracing/

³ Check “Two Generals’ Problem,”

https://en.wikipedia.org/wiki/Two_Generals%27_Problem

⁴ Full details in this article by Neha Narkhede, June 30, 2017, “Exactly-Once Semantics Are Possible: Here’s How Kafka Does It,” [www.confluent.io/blog/ex-](http://www.confluent.io/blog/exactly-once-semantics-are-possible/)

[actly-once-semantics-are-possible-heres-how-apache-kafka-does-it/](#)

5 Full walkthrough in “How to maintain message ordering and no message duplication,” <https://kafka-tutorials.confluent.io/message-ordering/kafka.html>

6 Full documentation in <https://kafka.apache.org/documentation/#semantics>

7 Detailed example in this article by Microsoft, 2012, “Exploring CQRS and Event Sourcing,” http://download.microsoft.com/download/e/a/8/ea8c6e1f-01d8-43ba-992b-35cfcaa4fae3/cqrs_journey_guide.pdf

8 Full article by Adam Warski, March 13, 2018, “Event Sourcing Using Apache Kafka,” www.confluent.io/blog/event-sourcing-using-apache-kafka/

9 Further details on why Kafka isn’t suitable for event sourcing in this article by Jesper Hammarbäck, August 18, 2020, “Apache Kafka Is Not for Event Sourcing,” <https://dzone.com/articles/apache-kafka-is-not-for-event-sourcing>

10 Further details on how to implement in this article by Rod Shokrian, October 1, 2019, “Event-Driven Architecture and the Outbox Pattern,” <https://medium.com/engineering-varo/event-driven-architecture-and-the-outbox-pattern-569e6fba7216>

11 Full article by Peter Alvaro, Neil Conway, Joseph M. Hellerstein, and William R. Marczak, January 9–12, 2011, “Consistency Analysis in Bloom: a CALM and Collect-Ed Approach,” <https://people.ucsc.edu/~palvaro/cidr11.pdf>

12 Very interesting article by Pat Helland and Dave Campbell, June 7–10, 2009, “Building on Quicksand,” https://database.cs.wisc.edu/cidr/cidr2009/Paper_133.pdf

13 Full article by Peter Bailis and Ali Ghodsi, April 9, 2013, “Eventual Consistency Today: Limitations, Extensions, and Beyond,” <https://queue.acm.org/detail.cfm?id=2462076>

14 Article on how to implement it by Robin Moffatt, March 13, 2019, “Kafka Connect Deep Dive – Error Handling and Dead Letter Queues,”

www.confluent.io/blog/kafka-connect-deep-dive-error-handling-dead-letter-queues/

[15](#) Full article by Ning Xia, February 16, 2018, “Building Reliable Reprocessing and Dead Letter Queues with Apache Kafka,” <https://eng.uber.com/reliable-reprocessing/>

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)